

PULSE

An AI analytics layer over a live clinical Postgres

Architecture, Progression, and Limitations

What we tried, what failed, what we chose, and what is still wrong with it

Status	In production. Owner-only surface.
Backend	health-hub-backend/src/services/pulse/ — 6,442 LOC TypeScript
Frontend	health-hub/src/components/pulse/ — 1,127 LOC React
Harnesses	21 suites, 2,867 LOC
Database	Neon Postgres, role <code>analytics_ro</code>
Model	DeepSeek, JSON mode, thinking off

Contents

Executive summary	3
1 The problem	4
1.1 What the system sits on	4
1.2 What makes this hard	4
1.2.1 The owner does not speak like an analyst	4
1.2.2 The dominant failure mode is silent	4
1.2.3 It cannot check its own work	5
1.3 What “good” means here	5
1.4 How to read this document	6
2 The progression: what we tried	7
2.1 The shape of the sixteen days	7
2.2 Era 0 — the offline architecture search	7
2.2.1 What was varied	8
2.2.2 What the numbers said	8
2.2.3 The three things this era actually established	8
2.3 Era 1 — V1, a router over fixed intents	9
2.4 Era 2 — replaying the audit log	10
2.5 Era 3 — V2, and the two keyword ladders	11
2.5.1 Ladder one: choosing the chart by matching the question	11
2.5.2 Ladder two: the same mistake, one file over	11
2.6 Era 4 — the analysis spec and its verification	12
2.6.1 The failure it was built for	12
2.6.2 The seven corrections that followed	12
2.7 Era 5 — investigation instead of “is that enough?”	13
2.7.1 Three things this got right	13
2.7.2 Three things this got wrong and had to fix	14
2.8 Era 6 — grounding: where does this number come from?	14
2.8.1 What was there before	14
2.8.2 What replaced it	15
2.8.3 Four corrections, all of them false positives	15
2.9 Era 7 — parse the SQL, do not grep it	16
2.10 Era 8 — binding: stop verifying backwards	16
2.11 Era 9 — money has a grain, and the grain is not optional	17
2.12 Era 10 — the missing place for arithmetic	18
2.12.1 The revert	18
2.13 Era 11 — derive, do not declare	19
2.14 Everything that was tried and rejected	20
3 The architecture we chose	22
3.1 Four principles, each the negation of a measured failure	22
3.2 The pipeline end to end	23
3.3 Ingress: precedence, and why it is not a ladder	23
3.4 The knowledge layer	24
3.4.1 Three decisions worth defending	25
3.4.2 Two structures that carry the semantics	25

3.5	The analyst: three prompts, three jobs	25
3.6	Spec, binding, and what each is for	26
3.6.1	How a scope constraint is resolved	26
3.7	The toolbox	27
3.8	Execution and the guard cascade	28
3.9	The SQL path	29
3.9.1	The validator	29
3.10	The investigation loop	30
3.10.1	The exits, and what each means to the owner	30
3.10.2	The one-step exit, and the trap in it	30
3.11	Arithmetic over evidence	31
3.12	Presentation: capability, then contract	31
3.12.1	What can truthfully be drawn	31
3.12.2	How much information goes where	32
3.13	Writing, checking, repairing	32
3.13.1	What <code>checkAnswer</code> enforces	33
3.13.2	When a card earns its place	33
3.14	Artifacts as conversational objects	34
3.15	The trace	34
3.16	The frontend	34
3.17	Cost, latency and limits	35
3.18	The test estate	36
4	Limitations, and where we need to improve	37
4.1	L1 — there is no degraded mode	37
4.2	L2 — generated SQL is not repeatable	38
4.3	L3 — binding is Phase 1 only, and Phase 2 is the whole point	38
4.4	L4 — the registry filters on eight columns; the schema has hundreds	39
4.5	L5 — grain is enforced by name, not carried as data	40
4.6	L6 — roughly a dozen regexes remain in load-bearing positions	40
4.7	L7 — materiality is the model’s own label on its own hypothesis	41
4.8	L8 — nothing measures whether the answer was useful	42
4.9	L9 — run-to-run variance is no longer quantified	42
4.10	L10 — the knowledge build runs against production	42
4.11	L11 — everything in the knowledge layer is Sobhana’s	43
4.12	L12 — the latency ceiling is honest but it is still two minutes	43
4.13	L13 — the frontend is the least-tested surface	44
4.14	L14 — Pulse answers, it cannot act	44
4.15	L15 — known open issues in the data itself	44
4.16	What to do next, in order	44
A	Reference tables	46
A.1	The metric registry	46
A.2	The eight filterable dimensions	46
B	Catalogue of named failures	47
C	File map	49

Executive summary

Pulse answers business questions typed in the owner’s own words — English, Hinglish, or a four-word fragment — against the same live Postgres the clinic runs on. It plans an analysis, executes it through a mixture of deterministic registry tools and generated SQL, decides for itself whether it has learned enough, and writes an answer whose every figure can be traced back to a step that produced it.

The system went through eleven distinguishable architectures in sixteen days. The headline finding of that work is short enough to state in one sentence:

The single measured result that shaped everything

Across 41 architectures and roughly 9,000 live model calls, **grounding was the only lever that ever moved accuracy**. Three independent post-hoc verification mechanisms — three-sample execution voting, LLM answer inspection, and an automatic repair loop — were each measured at approximately **zero**. The errors are systematic, not diverse: sampling the same wrong reasoning three times produces the same wrong answer three times.

Everything in the current architecture follows from that. The system does not try to catch its own mistakes by asking itself twice. It tries to make the mistake impossible to make, by moving every decision from something *declared* to something *derived*: derived from the rows, from the parsed SQL, from the database’s own grant tables, from arithmetic executed in TypeScript rather than written in prose.

The second finding, which cost more to learn, is that **an invariant described in a prompt is not an invariant**. Every rule this system enforces was first written as an instruction to the model, and every one of them was then violated by the model, in production, on a question that mattered. The rules that survive are the ones that became code.

Where it stands. On the last complete measurement: end-to-end chain 12/12, adversarial 28/29, pure layers 115/115, deterministic core 65/65, judge 12/12, frontend 7/7. A typical figure costs 3 model calls and 4–6 seconds; an investigation costs a median of 7 calls and up to 180 seconds, and discloses in the answer itself when it stopped early.

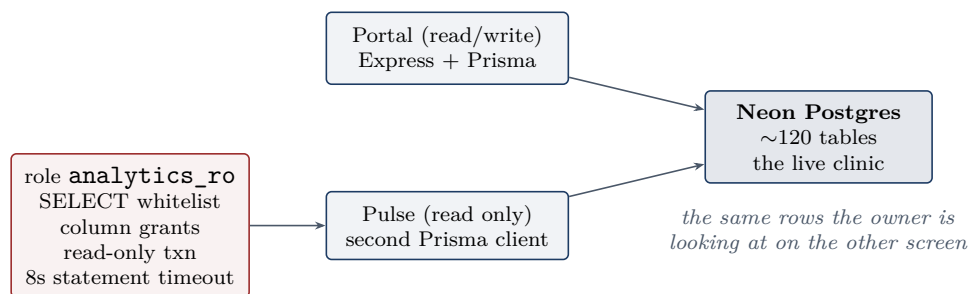
Where it is weak. Chapter 4 is deliberately the longest in this document. The short version: the system has no degraded mode when the model is unreachable; binding is complete for time and absent for scope, so every scope constraint is still verified backwards by parsing what the generator happened to write; roughly a dozen regexes remain in load-bearing positions and each has already been wrong at least once; and the suites that test whether the gears stay connected are the ones that cost money and are therefore run least.

Chapter 1

The problem

1.1 What the system sits on

Sobhana Diagnostics runs two branches (Chintal and Balanagar) with two more that exist only for testing. The portal that runs the business — registration, billing, test ordering, report authoring, WhatsApp delivery, referral payouts — writes to a Neon Postgres instance of roughly 120 tables. Pulse reads that same database through a second Prisma client bound to a restricted role.



The role is the real security boundary. It carries `SELECT` on a whitelist of tables, *column*-level grants on `Patient` and `User`, `default_transaction_read_only`, and an eight-second statement timeout. The SQL validator described in §3.9.1 is belt and braces on top of that, not instead of it.

1.2 What makes this hard

A natural-language interface over a database is a solved-looking problem with three properties that make it unsolved in practice on a business like this one.

1.2.1 The owner does not speak like an analyst

Measured directly: on *identical* control SQL, questions phrased in an analyst’s voice scored 20/20 and the same questions in the owner’s voice scored 16/20. Twenty points of accuracy sat entirely in phrasing. “Doctor” is genuinely ambiguous in this business — the referring doctor who *sends* the work and the clinic doctor who *sees* the patient are different tables, and “doctor wise” joined the wrong one. Hinglish (“kitna aaya”, “pichhle mahine”, “ab bhi zyada”) defeated a deterministic keyword router completely: 2 correct out of 6.

1.2.2 The dominant failure mode is silent

The failure that defines the problem

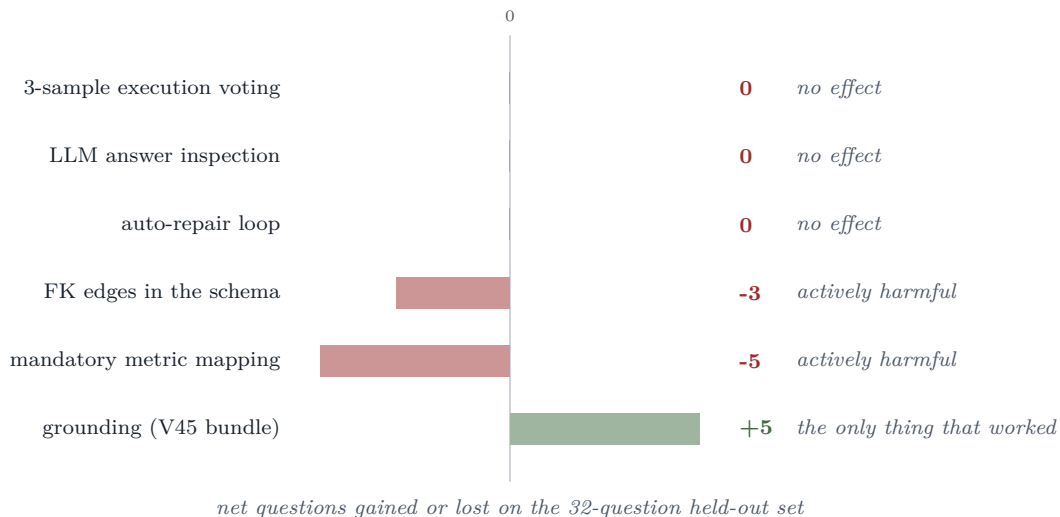
The most common error is not a crash, not a refusal, and not an absurd number. It is **a number that is wrong by less than 12%** — and therefore indistinguishable from a right one. It ships, it is believed, and it is acted on.

Every named failure in Chapter 2 has this shape. A centre-wide total labelled “Chintal”. A commission figure from the payout ledger presented as imaging commission. A bill total presented

as a test total. A paise figure spent as rupees. Each is plausible. None would be caught by a human glancing at the answer.

1.2.3 It cannot check its own work

Four self-correction detectors were built and measured. The best — agreement between two independent samples of the result — caught about 36% of genuine errors at a 5–10% false alarm rate. The automatic repair loop built on top of it fixed **zero** genuine errors across three full runs. LLM inspection of finished answers flagged between 9% and 27% of them and produced **no** score change.



Three mechanisms measuring zero is not three disappointments. It is one finding: the errors are *systematic*. Sampling a wrong chain of reasoning three times returns the same wrong answer three times, and asking the model to inspect its own output asks the same reasoning to catch itself.

1.3 What “good” means here

The bar is not “answers most questions”. It is a list of things the system must never do, ordered by how much they cost the owner:

1. Never present a figure for a wider question than was asked. A centre-wide total labelled “Chintal” is the worst output this system can produce.
2. Never state a number it cannot trace to a step.
3. Never report a limitation of Pulse as a fact about the business. “We do not record that” when the column has 44,743 rows in it is a lie about the owner’s own product.
4. Never present an incomplete investigation as a settled conclusion.
5. Never attach a card that adds nothing. Noise everywhere is the same as nothing anywhere.
6. Prefer silence to a confident wrong answer. Silence is recoverable.

1.4 How to read this document

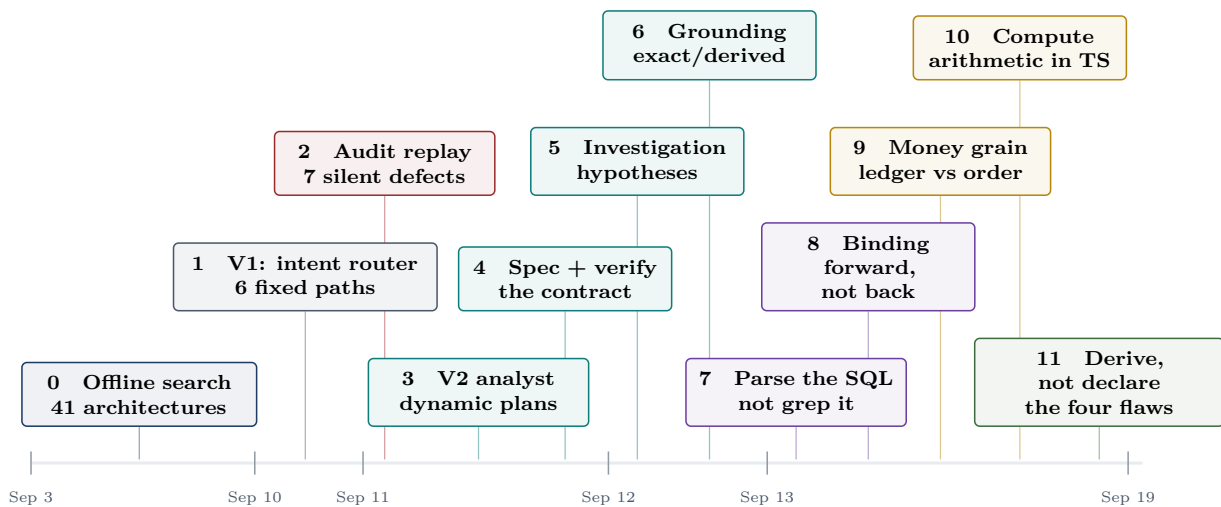
Chapter 2 is the progression: every architecture tried, in order, with what it measured and why it was kept, reverted or deleted. Chapter 3 is the architecture as it stands today. Chapter 4 is the limitations, at length. The appendices carry the reference tables — the metric registry, the toolbox, and a catalogue of every named failure.

Chapter 2

The progression: what we tried

This chapter is in chronological order because the order is the argument. Almost every design in the current system exists as the negation of something that was built, measured, and found wanting. Reading the end state alone makes it look over-built; reading it in sequence makes it look barely sufficient.

2.1 The shape of the sixteen days



2.2 Era 0 — the offline architecture search

Sep 3–10, 2026. 41 architectures, ~9,000 live model calls, 123 questions.

Before any product code existed, the question was which *kind* of system to build. The search ran over combinations of context material and execution strategy against a fixed question bank, scored by comparison against hand-written control SQL.

2.2.1 What was varied

Axis	Variants tried
Schema representation	bare <code>information_schema</code> dump; dump + FK edges; <i>graph schema</i> (tables split into FACT and DIMENSION by row count, with the time column named)
Business knowledge	none; metric registry; owner glossary; business ontology; join/grain conventions; worked examples; “advanced patterns” block
Value knowledge	none; a live value index mapping every branch code, test code, department, payout category and doctor name to its column
Execution path	one model call writing one SELECT; a semantic layer as the <i>execution path</i> (queries forced through metric definitions); a semantic layer as <i>context</i> only
Routing	no router; LLM intent router; deterministic keyword router
Verification	none; 3-sample execution voting; post-hoc LLM inspection; automatic repair; 2-sample result-agreement detection

2.2.2 What the numbers said

Mechanism	Measured	Verdict
Semantic layer as <i>context</i>	the biggest single gain	KEPT
Semantic layer as <i>execution path</i>	lost questions outright	REJECTED
Graph schema (fact/dim split)	part of the +5 V45 bundle	KEPT
FK edges in the schema block	−3 of 3 reps, +583 tok/question	REJECTED
Metric registry	+18 questions on conventions the schema cannot imply; ~0 on definitions it can	KEPT (narrowed)
Owner glossary	part of the +5 V45 bundle	KEPT
Mandatory metric mapping (“test” <i>always</i> = <code>test_orders</code>)	−5 on one question class (253→248)	REJECTED
LLM intent router	routing 15/24 → 24/24	KEPT (later deleted, §2.5)
Deterministic keyword router	2/6 on real owner input	REJECTED
3-sample execution voting	22.0 vs 22.0 single sample, 3× tokens, 3.6× latency	REJECTED
Post-hoc LLM inspection	flags 9–27%, no score change	REJECTED
Automatic repair loop	0 genuine errors fixed, 3 runs	REJECTED
2-sample result agreement	catches ~36% at 5–10% false alarm	KEPT as a <i>flag</i> only

2.2.3 The three things this era actually established

1. The 95% was a saturated-test artifact

The best configuration answered ~95% of the original 123 questions in one model call. A held-out “ceiling set” of 32 questions in *unseen reasoning classes* scored **20/32 = 63%**. The benchmark had been measuring how well the system had been fitted to the benchmark. Every later suite in this project is written until it fails.

2. Run-to-run noise is ± 2 questions at $n = 32$

Roughly $\pm 6\%$. Every previous “this change gained a question” claim was noise. Only the mutation results (-5 to -7) survived the noise floor — which is why the rejections above are trustworthy and small positive claims are not.

3. Grounding is the only lever

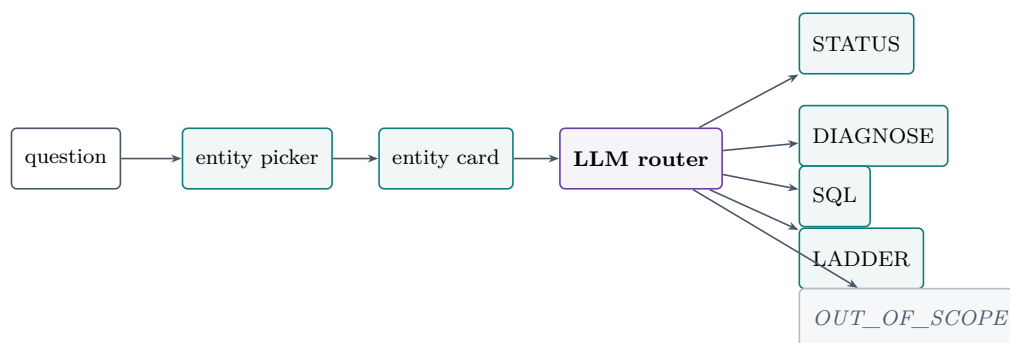
V45 — graph schema + owner glossary + join/grain conventions + LLM router — took the ceiling set from 20.0 to 25.0/32 with zero variance across two repetitions, owner voice from 16.5 to 18.0/20, routing from 15/24 to 24/24, behaviour from 15/25 to 25/27. **+5 questions for +16% tokens.** Every verification mechanism measured zero. The design rule that follows: *spend tokens on what the model is told, never on checking what it said.*

A fourth, smaller lesson, which has been re-learned twice since: **grounding must state defaults, not absolutes.** A mandatory mapping is read as a law and overrides the question. The glossary now says in as many words: “These are DEFAULTS, not overrides. If the question asks for something else — including cancelled rows, a raw row count, a different grain — follow the QUESTION.”

2.3 Era 1 — V1, a router over fixed intents

Sep 11. c4a1f45. Shipped to production.

The first build turned the winning offline configuration into a product. An LLM router classified each question into one of six intents and dispatched to a hand-built path.



What worked. The port scored *identically* to the offline harness on live data: held-out 25/32, owner voice 18/20. That equivalence is worth more than the score — it meant the harness was measuring the product and not a laboratory.

What broke it. The fallback. When the analyst path failed, V1 fell through to the SQL path, which had no idea what the conversation had been about. Asked “*how much of that is the second doctor*”, the fallback did not know what “that” pointed at and confidently answered a different question — using the pre-2c193be dues definition, including test branches, and none of the evidence the investigation had just gathered.

Rule taken from Era 1

A fallback that answers a *different* question is worse than no answer. Silence is recoverable; a confident wrong number is not. V1 was deleted entirely at ea5b634.

Bugs found in the port, all of one family. `query()`’s 200-row cap silently truncated the `information_schema` read, so the model was shown 200 columns and 200 doctors and never told. The FK query returned zero rows under `analytics_ro` — not an error, a zero. Both are the project’s signature failure: **a limitation presenting itself as a fact about the data.**

2.4 Era 2 — replaying the audit log

Sep 11. `e1415df, d4ccff3, 8433f00`.

Every Pulse question is written to `AuditLog` with `entityType='Pulse'`. Replaying that log against the database — asking, for each real question, whether the answer was true — found seven defects. **The 32-question benchmark stayed green throughout.**

Defect	What the owner saw
The writer could not see rows	<code>askResponse</code> projected evidence as <code>result: e.summary</code> and dropped <code>e.data</code> . <code>worklist</code> put its rows in <code>data</code> , so the writer told the owner to “pull the worklist rows directly”.
Money 100× too large	<code>t_query</code> put raw DB rows in <code>summary</code> , which the Evidence interface documents as “formatted strings, never raw paise”. Net billed reached the owner as 435,854,453 instead of Rs. 43,59,544.53.
A validator enforcing a wrong constraint	<code>completeSpec</code> trusted the planner’s <code>{dimension, value}</code> whenever the <i>dimension name</i> was valid. “ultrasound” arrived as <code>{test, 'ultrasound'}</code> ; <code>verifySpec</code> then <i>forced</i> <code>testCodeSnapshot = 'ultrasound'</code> into the SQL to satisfy its own contract, and the owner got a confident 0 .
A lexical miss became a claim about the schema	Asked which staff make the most mistakes, Pulse answered “there is no field anywhere in the system that records mistakes”. <code>AnomalyEvent</code> holds 44,743 rows with actor, role, severity and category, readable the whole time.
The dues flag disagrees with the arithmetic	The worklist filtered <code>paymentStatus <> 'PAID'</code> and the computed balance. On live rows 48 bills carry a non-PAID status while 10 actually owe anything. Reported 10 patients / Rs. 5,002 under a “complete list” headline; the truth was 11 / Rs. 5,802.
“scan” was undefined	It collapsed to every diagnostics test order: 5,104 against a true 528.
An exclusive end date, unmarked	Month-to-date counts whole days for like-for-like but handed the writer an exclusive end, so “1–11 September” silently excluded that day’s Rs. 15,650.

Rule taken from Era 2

A benchmark measures the questions you thought of. **Real usage measures the ones you did not.** Replay became a permanent instrument (`pulse-regression.ts`, then `pulse-replay.ts`), and every expected value in it is computed from the database at run time rather than hardcoded — which caught two bugs in the harness itself on its first run.

Two of these produced rules that are now load-bearing: `writerRows()` is *the one projection* into the writer’s hands, and `moneyCols()` reads which columns are money *off the SQL*, because column names lie — the generator emits `netBilledAfterDiscount` for a paise value with nothing in the name to say so.

2.5 Era 3 — V2, and the two keyword ladders

Sep 11. a87871d (analyst), a3b32b8 and 67ce15d (the ladders).

The six fixed intents were deleted and replaced by one general analyst that writes its own plan over a toolbox. That much worked immediately. What happened next is the most instructive episode in the project.

2.5.1 Ladder one: choosing the chart by matching the question

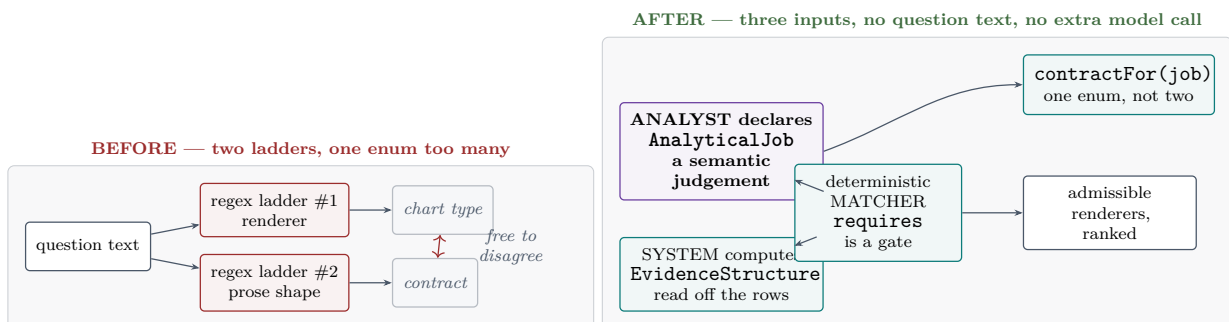
The first version of render selection was an ordered list of regexes over the question text. It failed **within an hour of being written**:

```
"break down discounts in the last 30 days by reason"
-> matched the TREND pattern (because of "last 30 days")
-> before the BREAKDOWN pattern was ever tested
-> a split question rendered as a time series
```

2.5.2 Ladder two: the same mistake, one file over

`deriveShape` was a second ordered ladder of regexes over the same question text, choosing the *prose* contract. It had the identical bug for the identical reason. Deleting the first and leaving the second would simply have relocated the failure.

There was also a *second enum*: a question had a SHAPE for prose and the plan carried a JOB for rendering. Two names for one idea, maintained in two places, derived by two different mechanisms, free to disagree.



The central rule of the architecture, learned twice

Nothing may match words against the question to decide how to answer. The analyst declares a semantic judgement (what is the owner trying to understand), the system computes a structural judgement from the rows (what shape is this evidence), and a deterministic matcher combines them. Models are good at the first and bad at the second; code is the reverse.

Measured effect of the contract layer. “Where am I losing money” went from **26 numbers in 5 sentences** to **4**. The budget that mattered turned out to be numbers, not sentences: three short sentences naming four branches and their four deltas is still the wrong answer.

A related trap, hit twice with the same shape. Structure was first derived from column *names*. `toNum` stripped non-digits before parsing, so “R0” became 0 and every text column read as a measure; and dimensions were detected from a name whitelist that contained “reason” and still scored zero dimensions. Both fixed the same way: **read the values, never the names**.

2.6 Era 4 — the analysis spec and its verification

Sep 11–13. 5fa8c9a, then a dozen corrections.

2.6.1 The failure it was built for

“*Last week’s lab collection at Chintal*” was answered with the centre-wide total and labelled “lab collection”. Every heuristic guard tried against this caught some cases and missed others, because they all asked “does this question *look* scoped?” rather than “did the query do what we said we would do?”

AnalysisSpec makes it a contract. The analyst writes down the measure, the scope and the period before any SQL exists, carrying the owner’s own word alongside the resolved literal:

```
{ goal: "lab collection at Chintal last week",
  measure: { concept: "collection", metric: "revenue" },
  scope: [ { term: "lab", dimension: "domain", value: "DIAGNOSTICS", how: "glossary", confidence: 1 },
            { term: "chintal", dimension: "branch", value: "CNT", how: "value-index", confidence: 1,
              aliases: ["chintal", "CNT"] } ],
  time: { phrase: "last week", period: "last-7-days", from: "2026-09-06", to: "2026-09-13" } }
```

A constraint present in the spec and absent from the SQL is a *rejection*, not a warning, and the repair prompt is told exactly which one went missing.

2.6.2 The seven corrections that followed

The spec was right and its verification was wrong seven separate times. This sequence is the best illustration in the project of how much surface a single good idea has.

Commit	The hole	The fix
8433f00	The spec trusted the planner’s value, so a validator <i>forced</i> a filter matching nothing	Ground scope terms through <code>resolveTerm</code> first; the planner’s guess is only the fallback
6c23e53	A string search for the literal is not a check that it restricts the answer	Parse the SQL and walk outward from the result-producing statement (§2.9)
627354b	A predicate behind an outer join restricts nothing	The walk respects join semantics
dee8474	The period was read from the plan, not from the query that ran	Period is derived from the SQL’s own time predicate
4911ec7	<code>verifySpec</code> guarded generated SQL; nothing guarded the registry tools	The same constraint check runs on registry steps, with an excuse for what the registry genuinely cannot express
2b1e54f	A constraint too weak to <i>filter</i> on was still used to <i>label</i>	If no step restricted the rows, the prose may not claim the scope
8fd794f	A tool could accept a filter and silently not apply it	Applied-scope verification: a tool reports the scope it applied; silence is refusal

The last one is the most important. `t_trend` took filter: `{modality: 'CT / MRI'}`, built its own `WHERE` clause without it, and returned the centre’s monthly totals — Rs. 18,61,499 for August against Rs. 1,32,500 of real CT billing. Every guard above it *saw the argument and approved*. Nine rounds and thirty-three model calls then went into reconciling a fourteen-fold contradiction the pipeline had invented, and the answer came out at 50 months against a true 76.

Rule taken from Era 4

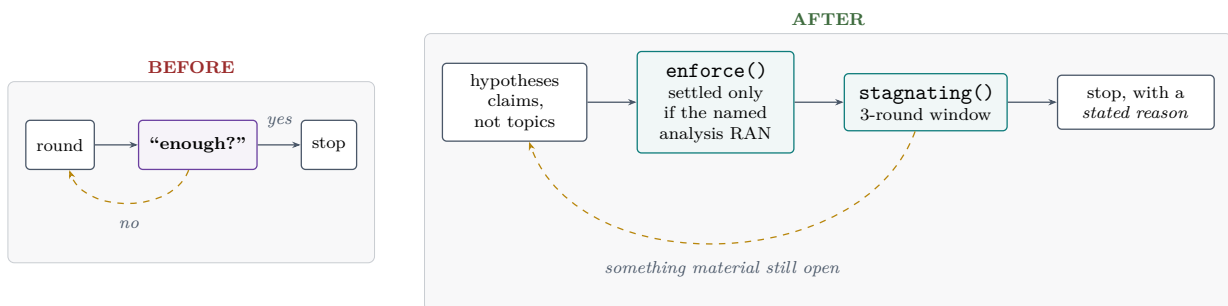
Checking that a constraint was *passed* is not checking that it was *applied*. A tool that applied a scope says so, in the scope it reports back; one that says nothing did nothing, whatever it was handed. This is a structural guard, so it catches the *next* tool to make the same mistake — which is the only way a class of bug stays fixed.

2.7 Era 5 — investigation instead of “is that enough?”

Sep 11–12. 62516d0, 5bc4882, 0e9d9d7, 2d29b43.

The first loop asked the model, each round, “*is this enough?*” and continued while it said no. That is a vibe check, and it produces answer generation rather than analysis: the model stops as soon as it has a number.

A real analyst does not stop when it has a number. Asked why collections fell 12%, it forms an objective, proposes the ways a decline can happen — branch, referrer, service mix, volume versus realisation, one-off events — and spends queries closing them off. “70% of the fall is two branches” does not end the investigation; it opens the next one, because branch-driven and volume-driven are different claims and only one more query separates them.



2.7.1 Three things this got right

Hypotheses must be claims. “The fall is volume, not realisation” is testable and is a *different* claim from “the fall is two branches”. A topic is not a hypothesis and a proposed step that resolves no named hypothesis is rejected as thoroughness.

A claim is settled only when its evidence ran. `enforce()` compares each hypothesis’s declared `requires` against what actually executed. Without it the loop still stopped on the model’s own say-so — an empty `next` is “I am done” wearing a structured coat, and the controller, not the model, decides that.

Stopping has a reason, and the reason reaches the owner. `resolved / insufficient_evidence / stagnation / resource_limit`. Incompleteness travels as *data* on the answer object, not as a sentence the writer might forget, because the guarantee cannot depend on a regex recognising whatever words the model chose.

2.7.2 Three things this got wrong and had to fix

Mechanism	What went wrong
Hard cap of 3 queries	Produced shallow answers to questions that deserved an investigation. Replaced by a generous budget plus stagnation. REVERTED
Stopping on one barren round	A cap wearing a new hat. Stagnation is now judged over a 3-round window ; one barren round is normal, because the analyst may be changing approach.
The detector called recovery stagnation	A query that fails, is told what it dropped, and comes back with the evidence has <i>advanced</i> the investigation. Evidence now carries recovered , three lines below where the comment says recovery is progress (a7fe89b).
Contradictions taken at face value	A conflict between two figures means nothing unless they were comparable. <code>screenContradictions + comparable()</code> drop a “contradiction” between figures from different periods or populations, and demote an unverifiable one to an open question.

And one measurement that mattered. The round reserve — time held back to write the answer — was *assumed* to be 45 seconds. Across the logged traces the write-up takes **5 seconds at the median and 10 at the worst**, on any job. The reserve was three times what it protects and every second of it was taken from the analysis: the loop stopped at 150s to leave room for something that finishes in ten. It is now 15s. *Measure the thing before budgeting for it.*

2.8 Era 6 — grounding: where does this number come from?

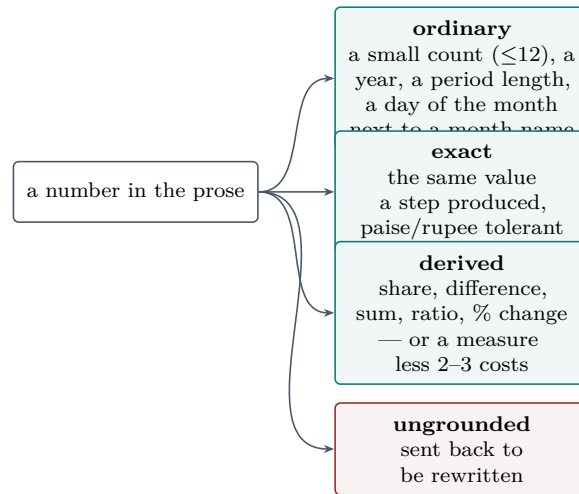
Sep 11. 69c1ec6, then ced089d, e1a3e4b, 8be613c.

2.8.1 What was there before

A proximity test: *does this number appear somewhere in the evidence, within 1%* — over a pile that also contained every value multiplied and divided by a hundred, to tolerate paise. That is an enormous equivalence class. It caught the blatant case (a fabricated Rs. 12,79,750) and could say nothing at all about a bare 72.

Worse, it could never catch the interesting error: **every number present, the relationship between them invented.** “Rs. 76,275 is 72.6% of Rs. 1,05,035” contains three real figures and one claim, and only the claim can be wrong.

2.8.2 What replaced it



Derivation is deliberately shallow — two-term arithmetic over the facts of one turn, plus the column totals the rows unambiguously entail. Searching deeper would start “proving” coincidences, and a grounding check that accepts almost everything is the proximity test again with more steps.

2.8.3 Four corrections, all of them false positives

A grounding check that flags a correct answer is worse than useless: it sends a true answer into a repair loop that can only make it worse.

Symptom	Cause and fix
“Contribution” flagged as invented	<i>Revenue minus commission, discounts and refunds</i> is four figures, all on the table, and the pairwise check tried $a - b$ and gave up. Now tries two and three deductions — the exact shape of payback, margin and contribution.
“May” triggering a period violation	Matched case-insensitively with the other eleven months, it fired on “these are not concepts that may appear”. May now counts as a month only with a year or a day number beside it.
A date read as an invented figure	“Chintal’s lab collected Rs.2,71,520 last week (6–13 Sep)” was flagged for inventing the number 13. Any 1–31 sitting next to a month name is the date it looks like.
The repair step could not read its own instruction	Indian money is written Rs.19,26,897, so a comma-joined list of flagged figures read as one run-on number. The repair — itself a model reading that sentence — was being asked to remove figures it could not identify. Separator changed.

Rule taken from Era 6

A check is a product surface. Every false positive it produces is an answer the owner does not get, or gets in a worse form. The bar for adding a check is not “does it catch the bad case” but “does it stay silent on every good one” — which is why the benchmark’s own judgement is itself under test (`pulse-judge.ts`, both directions, 12 cases).

2.9 Era 7 — parse the SQL, do not grep it

Sep 12. 6c23e53, 627354b, e4264d9.

verifySpec asked “does this constraint appear in the SQL”. That is a string search, and it is not the question. The question is whether the constraint restricts the rows the final number is computed from, and the two come apart exactly where being wrong is invisible:

```
WITH filtered AS (SELECT * FROM "TestOrder" WHERE "testCodeSnapshot" = 'CTBP')
SELECT SUM(o."priceInPaise") FROM "TestOrder" o
```

The literal is right there. The CTE is never referenced. The total is over every order ever placed — and a string search calls it correct. Nothing downstream would catch it: the number is plausible, it is genuinely grounded (a step really did produce it), and it is wrong.

v2/sqlscope.ts parses with pgsql-ast-parser and walks outward from the statement that produces the result, through only what it actually reads: base tables, *referenced* CTEs, subqueries, join conditions. Predicates found on that walk are *effective*; predicates anywhere else are decoration. It also grades how strongly a predicate pins a value down — *exact*, *set*, *contains*, *prefix*, *suffix*, *range* — because a LIKE is a real restriction but a looser one, and the difference decides whether it proves a scope constraint.

A parse failure is never a rejection

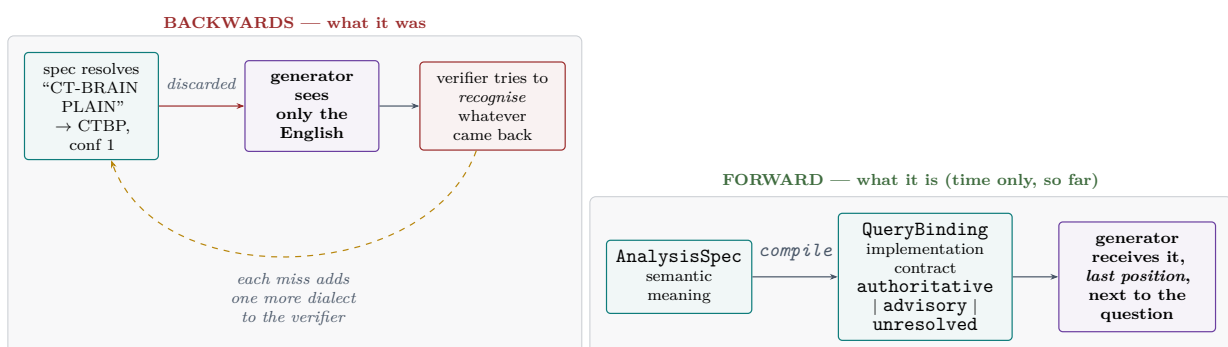
The generator writes Postgres this parser may not cover. A validator that rejects a query it merely failed to *understand* is a guard that can refuse a good answer but cannot confirm a bad one. On a parse failure the caller falls back to the string check — weaker, but never wrong in the strict direction.

The same parser fixed the security layer. The old PHI rule was “*PHI table mentioned and no aggregate → block*”, which conflated safety with authorization and got both wrong. It refused “the most recent CT-BRAIN PLAIN order and its amount” — a test name, a price, a date, no person in it — and told the owner the centre might not record it. It would equally have passed `SELECT name, phone FROM "Patient" GROUP BY 1,2 HAVING count(*) > 0`. `exposedIdentifiers()` now resolves every column reference to its table and blocks the identifying ones used *outside* an aggregate. A name inside `count(DISTINCT ...)` is a measurement; a name in the select list is a disclosure.

2.10 Era 8 — binding: stop verifying backwards

Sep 12. 1ca9e83.

By this point the verifier had learned aliases, then LIKE patterns, then predicate breadth, then derived-dimension sets. Each patch taught it one more *dialect* of a question that had already been answered upstream.



Two design choices are worth recording. `QueryBinding` is deliberately *not* the same type as `AnalysisSpec` and the spec gains no SQL fields — the moment a column name appears in `AnalysisSpec` the two jobs are merged again. And authority is a named type (`authoritative` / `advisory` / `unresolved`) rather than a confidence threshold, because a threshold is a number someone tunes and if (`binding`) `useBinding(binding)` is a regression nobody notices.

Phase 1 is time only, on purpose. Time has no joins, no aliases, no derived `CASE` expressions — it is the cheapest possible test of whether a typed binding survives the generator at all. Scope carries the whole DIMS problem (`br.code` presupposes a join; `modality` is a twelve-line `CASE`) and there is no point designing for it before knowing the mechanism works. **Phase 2 was never built**, and that is the single largest structural gap in the system today (§4.3).

2.11 Era 9 — money has a grain, and the grain is not optional

Sep 13. 718a4ca, b611a0f, 44d04bb, d7033a9, c780871.

This era was triggered by one question — “*does a Rs. 50,00,000 CT scanner pay for itself?*” — which the system could not answer for four days, in four different ways.

Basis error	What it produced	Fix
Commission from the pay-out ledger, scoped to a test	<code>DoctorPayoutLedger</code> has no <code>testOrderId</code> — it records what a doctor was <i>paid</i> . Filtering orders in a CTE and summing the ledger beside it counts commission for work outside the filter. Gave commission 4.5× and 6.6× the CT revenue it was supposedly derived from.	Split the metric in two: <code>commission</code> (ledger, scopeable by doctor/branch/period) and <code>commission_on_orders</code> (frozen on the order, scopeable by test/category/modality)
A bill total presented as a test total	A bill containing a CT scan also contains everything else on that visit. The same question answered Rs. 83,700 and Rs. 1,03,400 on consecutive runs depending on which grain the generator picked.	<code>TEST_GRAIN</code> typed repair, then refusal — a total at the wrong grain is not a smaller answer to the same question
Cash scoped to work	<code>revenue</code> is collected cash on <code>PaymentTransaction</code> and cannot be filtered by test at all. Asked what a CT scanner earns, the analyst reached for it, was refused, and answered “CT-scoped revenue has never been measured”.	Added <code>billed_on_orders</code> ; the refusal now <i>names the metric that can</i>
A ratio over two populations	Commission on orders billed in a window, divided by cash <i>collected</i> in that window. Both figures individually correct; the share wrong — 18.4% against a true 21.4%, because collection includes payments against bills raised months earlier.	The denominator of a commission share is what those same orders were billed
Rate presented as total	“Give me cost” for one test means the per-unit rate. The generator answered <code>SUM(o."priceInPaise")</code> over 47 orders and the writer reported Rs. 1,03,400 as what a single scan “is sold at”.	<code>RATE_NOT_TOTAL</code> typed repair

And the resolution bug underneath all of it. “*CT*” was *Clotting Time*. A three-letter live lab test code outranked the CT/MRI payout category on a substring match, so the whole

question filtered to the wrong thing. Fixed by **family disambiguation**: two terms in one question disambiguate each other through a transitive chain — test → payout category → service kind — so “ct scan machine” resolves *scan* to IMAGING first, and only then is it decidable that *ct* means the category (e248eb5).

Rule taken from Era 9

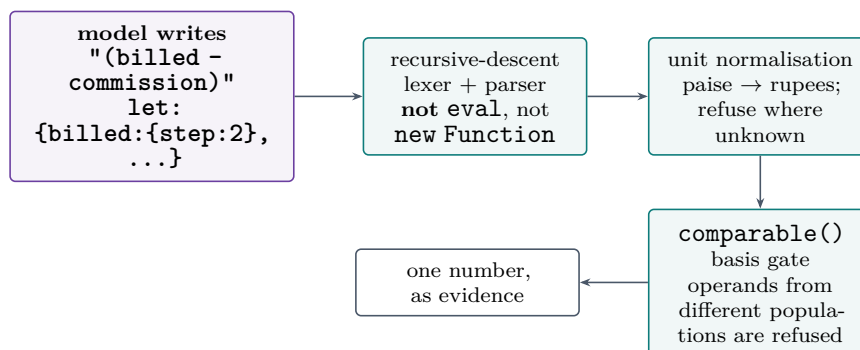
Same unit is not the same quantity. Every figure carries a *population* as well as a number, and two figures may only be combined when both are about the same one. A prompt cannot hold this — it was written into the conventions three different ways and the generator went on ignoring it. **Money needs enforcement.**

2.12 Era 10 — the missing place for arithmetic

Sep 13. b8269be, 851f9c8, 3a67fe8.

The writer is forbidden to compute — “*never divide or multiply a figure you were given*” — and that rule is correct: a model doing arithmetic in prose is how a figure lands 100× out as a real rupee claim. But the rule had no counterpart. Asked for the CT payback with CT revenue, CT commission and the Rs. 50,00,000 price all present and correct, the answer was “*I could not establish it*”. The operands were there. One subtraction and one division were missing, and there was nowhere for them to happen.

v2/calc.ts is that place: the model writes a formula and names its operands; the arithmetic runs in TypeScript over values read out of prior evidence, and the model reads the result back like any other evidence.



Money is the trap. Evidence carries money as *paise*; a price the owner typed is in *rupees*. Adding those gives an answer 100× wrong that looks entirely plausible — the worst kind. A generated query never declared its unit at all, so `capital / (billed - commission)` came out as 0.5 months. Fixed twice over: `t_query` now sets `unit: 'paise'` when money columns are the only numeric ones, and `calc` falls back to inferring from the column name (`/paise$|InPaise$/i`).

2.12.1 The revert

REVERTED Keeping the loop open a round so a compute step could run

Given another round, the investigation went exploring and bound “**calc**” — the owner’s shorthand for *calculate*, in “calc after removing referral amt” — to **CAL**, a three-letter lab test code, then filtered every query to it and returned nulls.

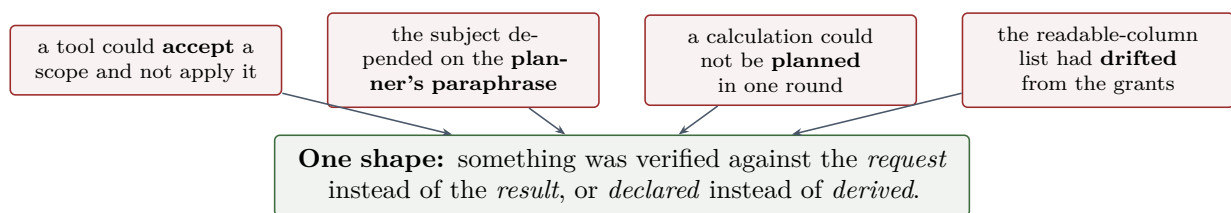
More rope is not the fix for arithmetic that did not happen.

The real fix was ordering. `compute` reads figures out of prior evidence, and steps in a round run in parallel, so its operands were required to come from an *earlier* round — which made a calculation impossible to *plan*: the planner cannot write `{"tool":"compute","let":{"billed":{"step":3}}}` before step 3 exists. So every calculation question measured its operands, took the one-step exit, and the writer did the division in prose. Ordering the round instead of forbidding the reference costs nothing: measurements still run in parallel with each other, and the arithmetic runs after them.

2.13 Era 11 — derive, do not declare

Sep 13. 8fd794f, 3a67fe8, 108b679, 6ee42fc, 32f8ab5.

The last architectural pass began with a diagnosis rather than a bug report. Four failures that looked unrelated turned out to have one shape.



Flaw	Was	Now
Scope	the guard read <code>args.filter</code> — the <i>request</i>	the tool reports the scope it <i>applied</i> ; silence is refusal
Subject	resolved against the planner's paraphrase: “CT scans” scored 0.66 on modality, bare “CT” scored 0.38 on the lab code	resolved against the <i>owner's own sentence</i> via a <code>termsIn</code> cross-reference
Arithmetic	operands had to come from a previous round, so the division happened in prose	measurements then arithmetic, <i>same round</i>
Column grants	a hardcoded <code>COLGRANT</code> list that had drifted; <code>Patient.name</code> was readable all along and the answer denied the centre records names	read from <code>information_schema.column_privileges</code> at build time

And one that is about the instrument, not the product. A benchmark run whose model credit expired halfway scored **7 of 24** — seventeen billing errors counted as quality failures, which reads as catastrophe and means nothing. The cause was that an unreachable model produced *exactly the same polite refusal* as a question the analysis genuinely could not answer. A refusal now carries whose fault it was (`reason: 'unavailable'`) and all three paid suites abort on seeing one rather than scoring the remainder.

Why a preflight was not enough

The first attempt was a credit check before the run. That proves the account works at call 1 and says nothing about call 30. The fix has to live where the failure happens.

2.14 Everything that was tried and rejected

Consolidated, because the rejections carry more information than the acceptances.

Mechanism	Measured outcome	Verdict
3-sample execution voting	22.0 vs 22.0 single sample; 3× tokens, 3.6× latency	REJECTED
Post-hoc LLM inspection of answers	flags 9–27%, zero score change	REJECTED
Automatic repair of flagged numbers	0 genuine errors fixed across 3 runs	REJECTED as correctness; kept only as <i>contract</i> repair
FK edges in the schema block	−3 of 3 reps, +583 tokens/question	REJECTED
Mandatory metric mapping	−5 on one question class	REJECTED — grounding states defaults
Deterministic keyword router	2/6 on real owner input; Hinglish defeats it	REJECTED
Semantic layer as execution path	lost questions outright; wins as <i>context</i>	REJECTED
Regex ladder for renderer choice	failed within one hour of being written	DELETED
deriveShape regex ladder for prose	identical bug, one file over	DELETED
A second enum (shape <i>and</i> job)	two names for one idea, free to disagree	DELETED
V1 fallback path	answered a <i>different</i> question with none of the evidence	DELETED
Per-step timeout	114s with it, 114s without; turned a slow-but-correct dues count into “nothing came back”	REVERTED
Hard cap of 3 queries	shallow answers to questions deserving an investigation	REVERTED
Stopping on one barren round	a cap wearing a new hat	REVERTED
45-second write-up reserve	measured at 5s median / 10s worst; cost 30s of analysis per turn	REVERTED (now 15s)
Loop kept open a round for arithmetic	bound “calc” to the CAL lab code and returned nulls	REVERTED
Blanket PHI refusal in the validator	refused a test name, a price and a date; would have passed <code>SELECT name, phone ... HAVING count(*)>0</code>	REVERTED — authorization is not a SQL-shape question
Credit preflight before a paid suite	proves call 1, says nothing about call 30	REVERTED
Proximity grounding (±1% over a ×100 pile)	caught the blatant case, blind to invented <i>relationships</i>	REVERTED
String-search verifySpec	passed an unreferenced CTE carrying the only filter	REVERTED (kept as parse-failure fallback)
Hardcoded COLGRANT list	drifted from the real grants; denied that names exist	REVERTED

Mechanism	Measured outcome	Verdict
Artifact rules stated in the writer's prompt	violated on 3 of 6 single-figure questions	REVERTED — enforced in <code>keepArtifacts</code>
Money-grain rules stated in the conventions	written three different ways, ignored	REVERTED — enforced as typed repairs

The meta-result

Of the mechanisms above, **every single one that asked the model to check itself measured zero**, and **every single rule that lived only in a prompt was eventually violated in production**. Those two sentences are the entire design philosophy of the system in Chapter 3.

Chapter 3

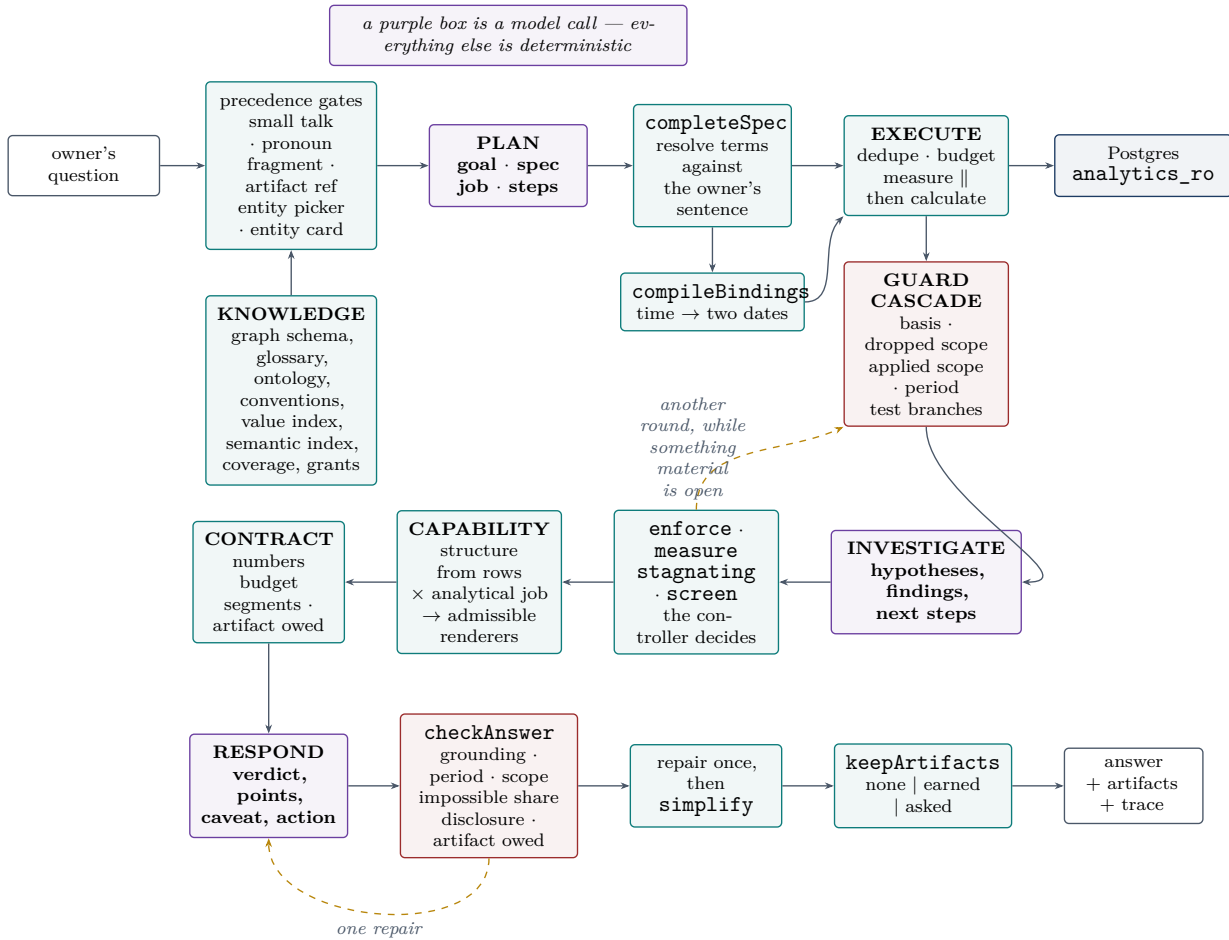
The architecture we chose

3.1 Four principles, each the negation of a measured failure

	Principle	The failure it negates
1	Ground, never verify. Spend tokens on what the model is told; never on asking it to check what it said.	Three verification mechanisms measured at zero. The errors are systematic, so a second opinion is the same opinion.
2	Derive, never declare. Every decision is read off the artifact it is about — the rows, the parsed SQL, the grant tables — not off a list someone maintains.	A hardcoded column list drifted and Pulse denied that patient names exist. A guard read the request instead of the result and approved a scope that was never applied.
3	Enforce in code what matters in money. An invariant described in a prompt is not an invariant.	Every money-grain rule was written into the conventions, some of them three different ways, and the generator went on ignoring all of them.
4	Never match words against the question to decide how to answer.	Two regex ladders, both dead within an hour, both for the same reason.

There is a fifth that is not a principle so much as a posture: **silence beats a confident wrong number**. Every refusal in the system says which of the two it is — a limit on Pulse, or a gap in the business — because reporting the first as the second sends the owner looking for data they already have.

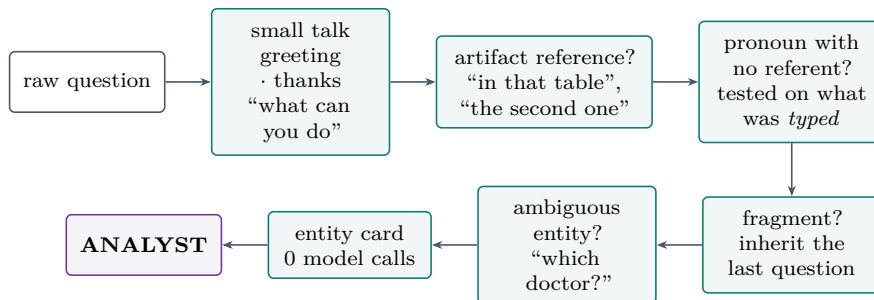
3.2 The pipeline end to end



Read left to right along the top, then right to left along the middle, then left to right along the bottom. The three purple boxes are the only model calls in the common case: **PLAN**, **INVESTIGATE** (skipped when one step answers the question), and **RESPOND**. Each query step adds one more, plus one per repair.

3.3 Ingress: precedence, and why it is not a ladder

services/pulse/index.ts. Five cheap deterministic checks run before the analyst. Their *order* is the design, and it was rearranged twice to fix real bugs.



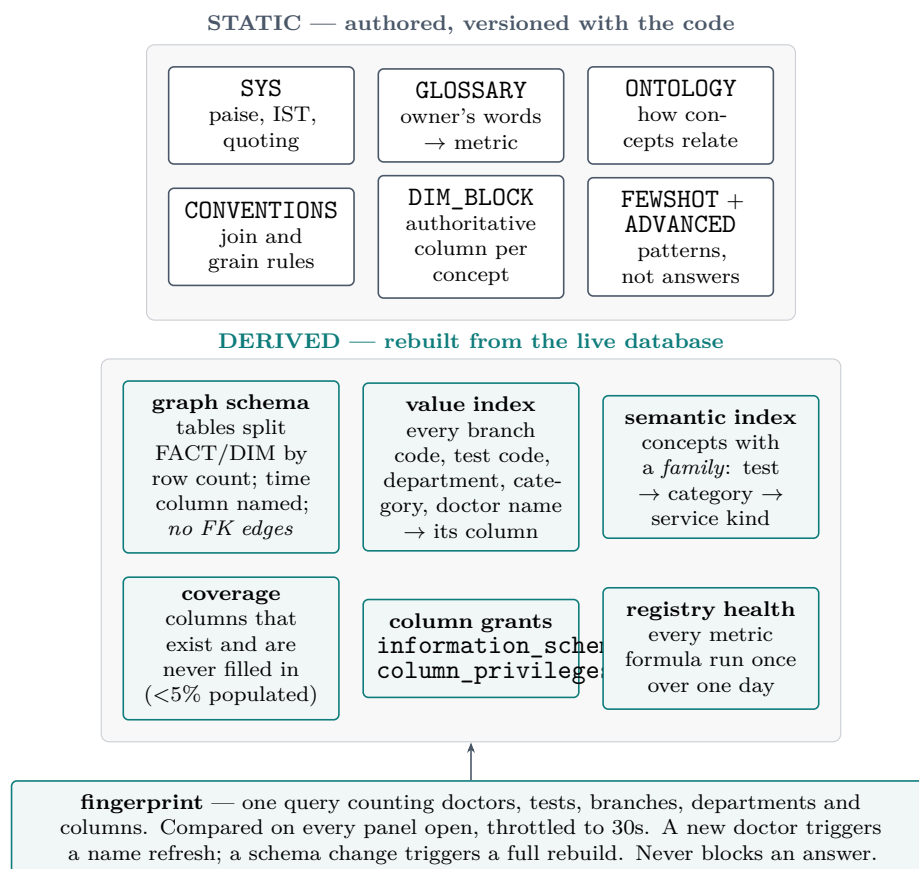
Why entity resolution is no longer the first gate. It used to be. The owner was shown a table of discount by referring doctor, pasted it back, and said “*in that table*”. Pulse answered “*Which mbbs?*” — twice — because the pasted text contained “MBBS” across several doctor names. “In that table” is a conversation problem, not an entity problem. An artifact reference now outranks entity resolution, and so does a pasted block (over 180 characters, or three or more rupee signs, or embedded newlines).

Why the pronoun guard tests the raw text. Asked “*name him*” straight after a collection total — an answer that named nobody — Pulse went and found the largest referring doctor of the day and presented him as the answer. It is grounded and it is not what was asked. Three lines later a fragment inherits the previous question and the pronoun is no longer alone on the line, so the check must run on what the owner *typed*.

Why a follow-up replaces a dimension rather than adding one. “Doctor wise” after “branch wise” is not $\text{branch} \times \text{doctor}$. The previous question has its dimension phrase stripped before the new one is appended.

3.4 The knowledge layer

`services/pulse/knowledge.ts` (928 lines) and `catalog.ts` (196). This is the part the Era 0 search said was the only thing that mattered, and it is built from live data at startup rather than written down.



3.4.1 Three decisions worth defending

No FK edges. Emitting foreign keys into the schema block cost 583 tokens per question and lost the comparison 3 reps out of 3. The model does not need to be told that `Bill.visitId` points at `Visit.id`; it needs to be told which of two doctor tables means “sent us the business”.

The registry is narrow on purpose. Fifteen metric formulas, each the *only* place that expression exists. Measured, the registry is worth +18 questions on conventions the schema cannot imply — the authoritative column, the stable date column, the cancelled-row rule, a `NULL` denominator — and approximately *zero* on definitions the schema already implies. Every formula in it is one of the former.

Hollow columns are stated, not discovered. A column that exists and is never populated returns a confident zero rather than an error. The build measures fill rates on a watch list and tells the model in as many words: *if a question needs one of these, say the business does not record it and ask the owner — do not substitute a different column and label it as the thing they asked for.*

3.4.2 Two structures that carry the semantics

METRICS	15 named quantities with a formula, a unit, a time column, a filter and a prose definition. Two of them exist <i>because</i> one name covered two real quantities: <code>commission</code> reads <code>DoctorPayoutLedger</code> (what a doctor was <i>paid</i> , scopeable by doctor/branch/period) and <code>commission_on_orders</code> reads the commission frozen on each order (scopeable by test/category/modality). A third, <code>billed_on_orders</code> , exists solely to give a scoped ratio a denominator at the same grain.
DIMS	8 filterable dimensions, each a SQL expression. Three of them are <i>rolled up</i> : <code>modality</code> folds “Ultrasound”, “Ultrasound Tiffa” and “2D Echo” into one value, because asked “how many ultrasound scans” a single-value filter on the raw column silently answers a third of the question.

Test branches. JGG and IDPL exist only for testing — 29 and 8 lifetime visits against Chintal’s 3,011. They are removed from every dimension *split*, where their movement reads as a business finding, and never from a centre-wide *total*, where silently changing what a total covers would be worse. The rule is enforced centrally in `runStep`, because six tools build branch-keyed rows independently and fixing one of them left JGG sitting on the next chart the owner looked at.

3.5 The analyst: three prompts, three jobs

`v2/analyst.ts` (490 lines). One general-purpose analyst replaced six fixed intent paths.

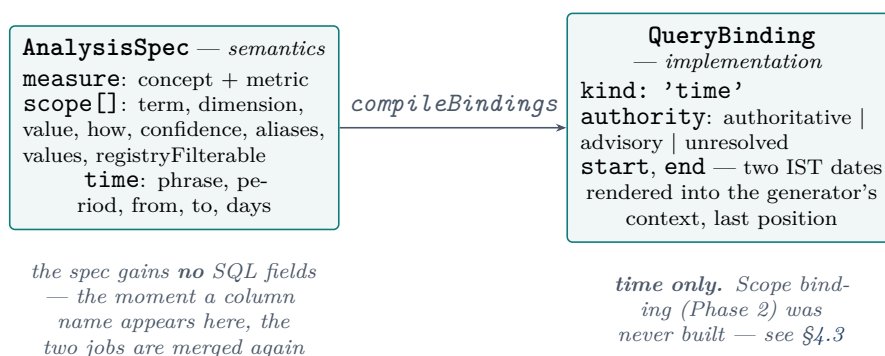
Call	Returns	Constrained by
PLAN	goal, spec, job, present, steps[], outOfScope, phi	the toolbox, the metric list with its splittable dimensions, the semantic index summary, the resolved terms of <i>this</i> question, and the previous turn's question and plan
INVESTIGATE	findings[], hypotheses[] with requires[], contradictions[], next[]	the evidence so far and the prior investigation brief. Its next is filtered to steps that resolve a <i>named open</i> hypothesis
RESPOND	verdict, points[], caveat, action, artifacts[], suggest[], opportunities[]	the response contract (how many numbers, which segments exist, whether an artifact is owed) and the <i>admissible</i> renderer list — it cannot ask for a chart the evidence will not support

Two things the analyst is explicitly *not* allowed to do. It may not compute — a model doing arithmetic in prose is how a figure lands 100× out — and it may not decide the renderer by naming one; **present** is resolved from the owner's words upstream and arrives here as a renderer *type*, never as a sentence.

Plan repair, before anything runs. Three deterministic rewrites sit between the plan and the executor:

1. A qualifier in the question that the *spec* failed to capture, and that no step covers, rewrites the whole plan to a single **query** step. This is the silent-drop failure: “only lab” answered with the total including OP fees, labelled “lab collection”. Validated filters catch a *wrong* filter; nothing caught a *missing* one.
2. A single registry step under a question carrying **median**, **per**, **never**, **each**, **distinct**, **except...** is rewritten to **query**. A registry tool computes one fixed thing; a close number is a wrong number.
3. A single **query** step gets the owner's words *verbatim*. The analyst's paraphrase drops qualifiers often enough to matter.

3.6 Spec, binding, and what each is for



3.6.1 How a scope constraint is resolved

completeSpec does not take the planner's word for it. The owner's term is resolved against the *question it sits in*:

```
const fromQuestion = termsIn(q) // every term the QUESTION contains
  .filter(t => t.dimension && t.value
    && t.phrase.toLowerCase().includes(firstWordOf(c.term)))
  .sort((a,b) => b.score - a.score)[0];
const direct = rankedIn(c.term, q) // the planner's term, resolved in context
  .find(r => r.score >= MIN_CONFIDENCE && r.dimension && r.value);
const hit = fromQuestion && (!direct || fromQuestion.score > direct.score) ? fromQuestion : direct;
```

This exists because the subject used to depend on the planner's paraphrase: “CT scans” scored 0.66 on the modality and bare “CT” scored 0.38 on CT, the live lab test code for Clotting Time. The same question resolved differently depending on how the planner chose to restate it.

Confidence gates enforcement, not resolution. A guess may *guide* a query; only a confident resolution may *gate* one. “CT-BRAIN PLAIN” once resolved to the lab code CT on a substring match and the validator then forced `test = CT` into three separate queries, turning one bad lookup into a whole turn of wrong answers.

Aliases are the same identity, not alternatives. A test is both a code and a name; the generator writes one or the other about half the time. A validator that knows only the code rejects a correct query — that alone was the whole of this question's non-determinism.

registryFilterable: false is an honest escape hatch. The registry filters on eight columns; the schema has hundreds. A constraint the registry cannot express is still a constraint the owner asked for. Dropping it from the spec meant nothing downstream knew about it, so “how much from external reports” answered with the centre's entire August revenue and no check could see that the scope had never been applied. It is now carried, excused on registry steps, and still enforced on generated SQL and on the prose.

3.7 The toolbox

`v2/tools.ts` (1,045 lines). Eighteen tools. Seventeen are deterministic and cost nothing; one writes SQL.

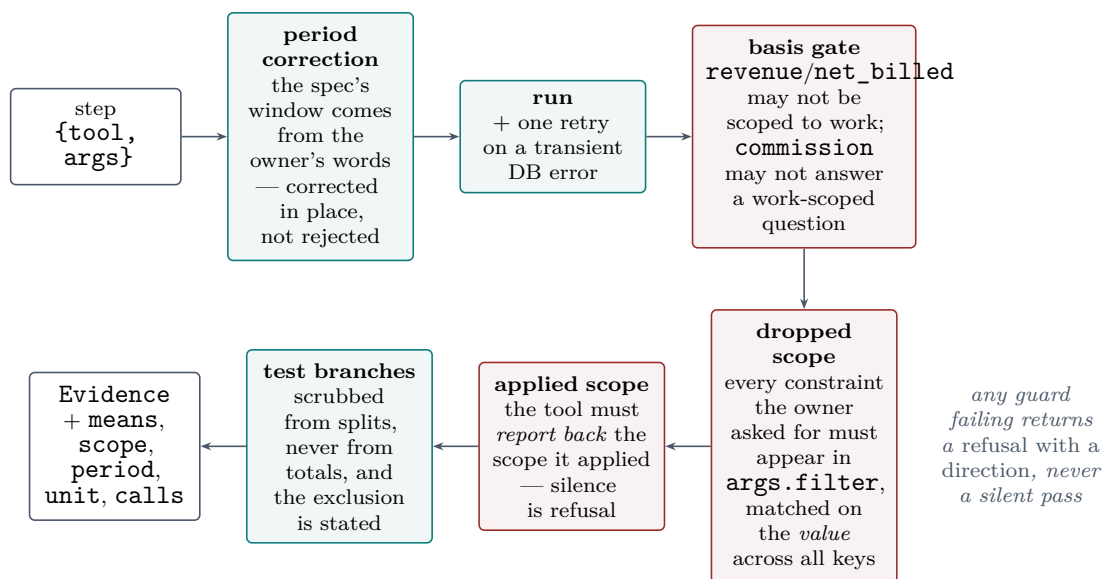
Tool	Cost	What it is for
<code>query</code>	1–3 calls	Writes SQL for exactly the question given, against the full schema. The accurate general tool and the right default for any specific figure.
<code>metric</code>	free	One registry figure. <code>{metric, period, filter}</code>
<code>compare</code>	free	The same metric over two periods, with the delta
<code>breakdown</code>	free	One metric split by one dimension, with parts and shares
<code>rank</code>	free	Top <i>n</i> of a dimension by a metric
<code>trend</code>	free	A metric bucketed over time
<code>baseline</code>	free	What normal looks like for this metric
<code>anomaly</code>	free	Points outside the baseline
<code>derive</code>	free	A ratio of two registry metrics at a shared grain
<code>compute</code>	free	Arithmetic over figures prior steps produced (§3.11)
<code>resolve</code>	free	What a word means in <i>this</i> business, from live data
<code>receivables</code>	free	Who owes, by branch and age
<code>pending_reports</code>	free	What is late, against the SLA
<code>quiet_doctors</code>	free	Referrers whose volume stopped
<code>leakage</code>	free	Discounts, refunds, cancellations, reversals
<code>worklist</code>	free	The owner's own rows — names, numbers, amounts (capped at 200)
<code>delivery</code>	free	WhatsApp delivery and read rates
<code>anomalies</code>	free	<code>AnomalyEvent</code> by actor, role, severity, category

The split is the point. A plan of six registry tools costs *zero* model calls, so the analyst can afford to look at six things before deciding what matters. Only **query** steps buy latency, and the loop budgets them explicitly.

A refusal names the tool that can. The registry’s error messages are written as directions rather than walls, because a dead-end error gets retried. Asked for revenue filtered by modality, the planner was refused, asked again the next round, and the next — ten of nineteen steps in one investigation were the same refusal, and the CT payback question ran out of budget without ever measuring CT revenue. The message now computes, from `METRIC_DIMS`, which metric *does* carry that dimension at the same unit — and ranks cost-shaped metrics last, because offering `commission_on_orders` first for a revenue question invites exactly the basis error this layer exists to remove.

3.8 Execution and the guard cascade

Each step passes through six guards on its way back. Every one of them exists because a specific wrong number reached a specific owner.

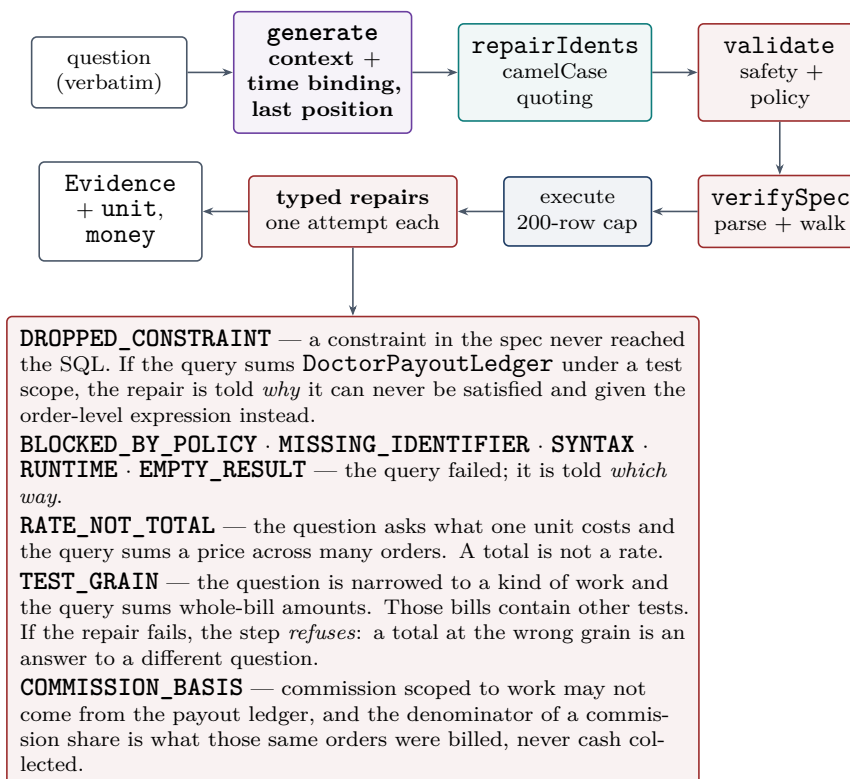


The period guard corrects rather than rejects. A step that merely fails here costs a round and comes back with the same guess. The spec’s period comes from the owner’s own words, so it is not a preference to be weighed against the planner’s. Before this, a step ran `last_30_days` under a spec that said 90 — and the writer, reading the question rather than the step, reported it as “21.4% over the last 90 days”. Every filter correct, the number correct for what it measured, and the sentence wrong.

The scope match is on the value, not the dimension. `test = CT` is a *guess* about which column carries “CT”. Filtering by `modality = CT` honours exactly what the owner asked for, and rejecting it blocked every route to a CT figure at once.

Ordering within a round. Measurements run in parallel (pool of 3); compute steps run afterwards, sequentially, seeing every figure the measurements produced. A step that already ran this turn — same tool, same arguments — does not run again. One investigation re-ran the identical monthly trend seven times and the same thirty-day CT query eight: thirty-three model calls where six would have done, and the answer was worse at the end than at the start.

3.9 The SQL path



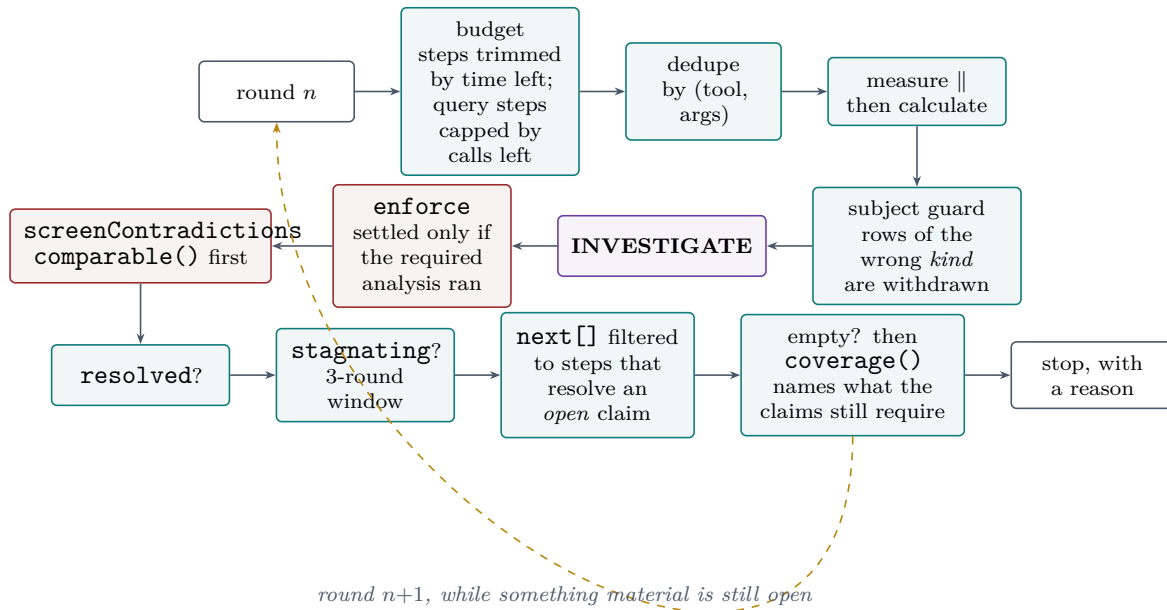
3.9.1 The validator

`validator.ts` is the deterministic safety layer, and it is itself under test: a conventional validator leaked 5,451 report bearer tokens, 14 password hashes and 30,581 patient results on the benchmark. This one blocks all four classes.

Always, for every-one	not a <code>SELECT</code> ; multiple statements; a write keyword; a system catalog; a table or column <code>analytics_ro</code> has no grant on; <code>SELECT *</code> ; a comma-join (implicit cross product); a <code>JOIN</code> without <code>ON</code> ; an aggregate mixed with a bare column and no <code>GROUP BY</code> ; fan-out — aggregating a parent column across a one-to-many join, which double-counts
Policy, from the role	<code>rowLevel === false</code> resolves every column reference to its table and blocks identifying ones used outside an aggregate. On a parse failure it falls back to the blunt rule, because this is the one guard where being wrong exposes a patient.

An explicit `CROSS JOIN` is deliberate syntax — calendar $\times$ dimension grids need one — and is bounded by the role's statement timeout. A comma-join is an accident and stays blocked.

3.10 The investigation loop



3.10.1 The exits, and what each means to the owner

Reason	Trigger	What the answer must say
resolved	nothing material is open	nothing; it is a complete answer
stagnation	3 rounds with no new evidence, no settled claim, no new dimension	the line of enquiry stopped paying; what is still open
insufficient_evidence	no proposed step, and none derivable from what the claims require	it could not be established — in the <i>verdict</i> , before any conclusion
resource_limit	40 steps, 12 rounds, 30 calls or 180s	the same, and this is a <i>failure</i> mode to disclose, not a normal finish

Incompleteness travels as *data* on the answer object. The contract check also nudges the prose, and on the run that failed the suite the prose happened to disclose it — but that is luck, and the guarantee cannot depend on a regex recognising whatever words the model chose.

3.10.2 The one-step exit, and the trap in it

A one-step plan that worked has nothing to interpret, so it skips INVESTIGATE entirely — the common case, and it saves a whole round trip. The trap: *one step succeeded is not one answer found*. A *resolve* step succeeds by telling you what a word *means*; it never carries a number. “How much am I making from external reports” planned a single resolve, got `EXTERNAL_UPLOAD` back — correct, fourteen hundred orders of it — broke out of the loop, and told the owner the figure could not be established. It resolved the term and then denied the thing it had just found. A step that measures nothing may not end the turn.

3.11 Arithmetic over evidence

v2/calcul.ts (217 lines). The model writes a formula and names its operands; the arithmetic runs here.

```
{ "tool": "compute",
  "label": "months to repay the scanner",
  "args": { "formula": "capital / (billed - commission)",
    "let": { "capital": { "value": 5000000, "unit": "rupees" },
      "billed": { "step": 2, "field": "billed_on_orders" },
      "commission": { "step": 3 } } } }
```

Recursive descent, not eval	The formula is model-authored text. Handing model-authored text to a JS engine is a remote-execution hole dressed up as convenience. The evaluator handles + - * / () and names, and nothing in it can call anything.
Operands find their own column	An operand names a <i>field</i> ; the evaluator matches it against the step's columns. Ambiguity is refused rather than guessed. Without this, all five compute steps in one run failed with “carries no single figure — name a field”.
Units are normalised or refused	Money is paise in evidence and rupees when the owner types a price. Every operand is normalised to rupees on the way in; where a unit cannot be established the operand is <i>refused</i> , not assumed.
The basis gate	<code>comparable()</code> runs over the operands' identities. Two figures from different periods, scopes or populations do not combine, whatever the formula says.

3.12 Presentation: capability, then contract

3.12.1 What can truthfully be drawn

v2/capability.ts (292 lines). `describeEvidence` reads a structure off the rows — `rowCount`, `cardinality`, `dimensions`, `measures`, `hasDeltas`, `partsOfWhole`, `isTimeSeries`, `hasStages`, `numericSpread`, `continuous`, `seriesPerEntity`, `hasTarget` — and ten renderer capabilities are tested against it.

Renderer	Hard gate (requires)	Buildable when
kpi	cardinality = one	a value, or exactly one row
compare	hasDeltas, cardinality = one	now and before both present
waterfall	hasDeltas, 1 dimension	≥2 rows carry a non-zero change
pareto	partsOfWhole, 1 dimension	≥3 rows with a positive value
breakdown	1 dimension	≥2 labelled rows
ranking	1 dimension	≥2 labelled rows
chart	isTimeSeries	≥3 points
funnel	hasStages	≥2 stages
distribution	continuous	≥6 rows
table	1 dimension	≥1 row

requires is a gate, expresses is a ranking

An unsatisfied **requires** means **not a candidate** — never a low score. The previous design checked only that the *string* “waterfall” appeared in the response, and the waterfall renderer returns null when no row carries a delta, so a *required* artifact could ship rendering nothing at all and validation called it valid.

buildable is a second, stricter gate: can the frontend actually construct this renderer's input from this step? Structure alone was not enough — a breakdown bound to a query step passed **dimensions: 1**, looked for **summary.parts**, found the rows under **summary.rows**, drew nothing, and still painted a titled empty card.

3.12.2 How much information goes where

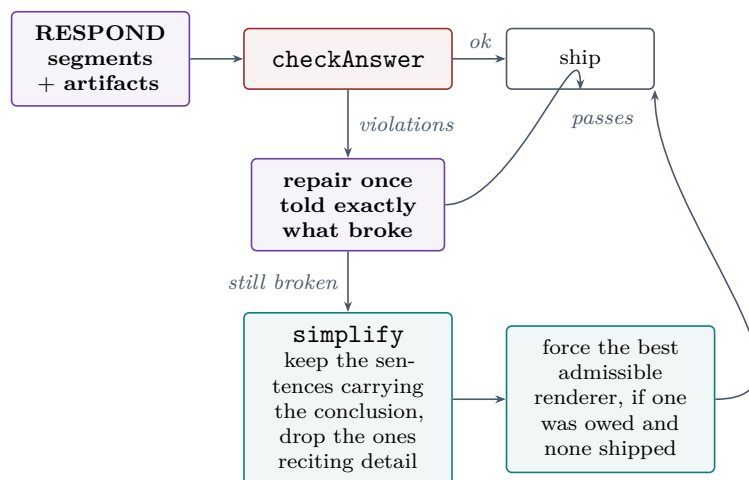
`v2/contract.ts` (344 lines). The contract is keyed by the analytical job and tuned by the evidence structure. **No question text reaches this file.**

Job	max numbers	points	artifact	what prose is for
magnitude	3	0	no	the number, its scope, its period
comparison	5	2	no	both figures and what the change means
composition	6	2	yes	the total and the parts that carry it
concentration	6	2	yes	how few things account for how much
attribution	8	3	yes	what moved and what drove it, ≤ 3 drivers
progression	5	2	yes	direction, size, turning point
variability	5	2	yes	the typical value <i>and</i> the tail
conversion	5	2	yes	who entered, who came through, where the loss is
relationship	6	2	no	whether they move together, and who breaks the pattern
ranking	6	2	yes	who is top and by how much
enumeration	5	1	yes	<i>how many</i> and <i>what they total</i> — never the rows
explanation	8	3	no	a real explanation; prose <i>is</i> the right answer here
opportunity	—	—	—	ranked levers, largest realistic value first

The budget is *numbers*, not sentences. “Branch A Rs. 83K, B Rs. 61K, C Rs. 43K” is a bad answer even in one sentence. And the question outranks the job in exactly one direction each way: an explicit “can I see” forces an artifact whatever the job thought, and an explicit “no table” forbids one.

“No table” is not a request for a table. Matching the bare word turned the strongest signal there is — the owner saying *don’t* — into the opposite instruction, and “just tell me the number, no table” shipped a KPI tile. A negation near the verb outranks it, and an explicit no is final.

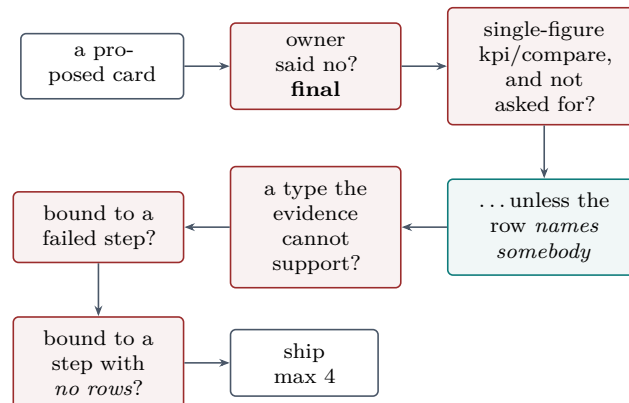
3.13 Writing, checking, repairing



3.13.1 What `checkAnswer` enforces

Check	The answer it exists to stop
Numbers budget	More figures than the job allows, or any single sentence carrying more than three
Period grounding	<i>“CT-BRAIN PLAIN has been billed Rs. 1,03,400 for the period 1–13 September”</i> — the figure is all-time and the SQL restricts no time column at all. A correct total under a wrong window is a wrong answer.
Scope grounding	<i>“At the current CT run-rate of Rs. 18,93,725 a month”</i> — that is the <i>centre’s</i> August revenue. A constraint too weak to filter on is too weak to label with.
Impossible share	<i>“discounted at 1,250% of what it bills”</i> . A part cannot exceed its whole; the shape comes from dividing a bill-level discount by one test’s price. Tested on the <i>shape</i> , comma-aware, because matching exact phrasings loses to the next sentence the writer chooses.
Grounding	Any figure that is neither exact, nor derivable from two that are, nor ordinary
Disclosure	An investigation that did not close, presented as settled — and it must be said in the <i>verdict</i> , not a trailing caveat
Artifact owed	The job wants one, the evidence can support one, and none shipped

3.13.2 When a card earns its place



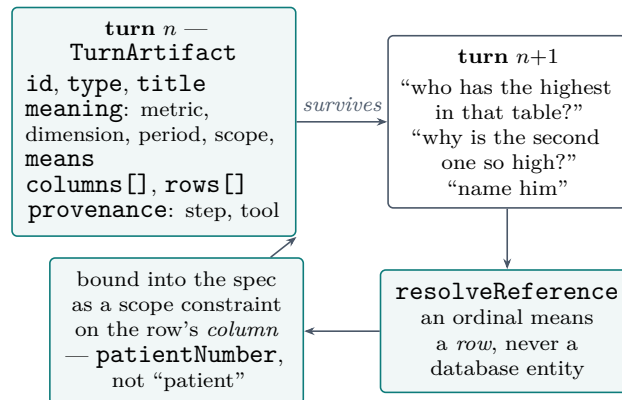
Why the identity exception exists. The card is not only a picture; it is what the next question points *at*. “Who is the most repeating customer” answers with one figure, so the single-figure rule dropped its card — and “name him” then had no row to resolve against and came back with nothing. A single row carrying a patient number or a name is the anchor for the follow-up. One number with nobody attached is the noise.

Why this is code and not a prompt rule. It *was* a prompt rule — “never attach an artifact that only repeats a single figure the sentence already gave” — and the writer attached one anyway, on three of six single-figure questions.

Why the empty-card check uses the judge’s own `rowsOf`. The first version wrote its own row detection and the two disagreed: the product shipped a card it believed had rows and the benchmark flagged it as drawing nothing. Two definitions of “has rows” is one too many.

3.14 Artifacts as conversational objects

An artifact stops being a render instruction and becomes an object that survives the turn, carrying what it *means*.



The row the owner pointed at is already known — it is on screen — so it becomes a scope constraint that `verifySpec` enforces like any other. The analyst may still choose *how* to answer; it may not change *who* the answer is about. A step that returns rows of a different *kind* is withdrawn mid-round, with an explanation, rather than reconciled.

Binding the column, not the type. “Patient” is a kind of thing; it has no entry in DIMS and no column behind it, so a constraint written that way is one the generator cannot translate and the repair loop then mangles. `patientNumber` is a column that exists, and the literal survives into the query — which is the whole point.

3.15 The trace

Every turn writes a structured record of every decision it made: the plan and its spec, each executed step with its SQL (4,000 chars — at 600 two thirds of recorded queries were cut mid-expression), its *summary values*, its period, scope and unit, the investigation with its hypotheses and progress history, the render structures and admissible options, the contract, the violations before and after repair, the grounding verdict for every figure in the prose, and the timing split between evidence and write-up.

Why the values, and not just the row count

The trace originally recorded the SQL, the tool and the row count but never the values — so re-judging a recorded answer found no facts to ground it against and reported **19 of 26** as inventing numbers. Every one of those was the absence of the evidence, not the presence of a lie. A trace that says how many rows came back but not what they said cannot settle whether a figure in the prose was real, which is the whole reason anyone opens one.

3.16 The frontend

`health-hub/src/components/pulse/` — an orb, a floating panel, a feed of cards (not chat bubbles), and an evidence drawer.

One derivation, shared by every renderer. `deriveView.ts` replaced `seriesOf()`, which guessed which column was the label and which was the value and threw the rest away. Measured across the artifact types, the renderers were showing **85%** of the material information they were handed, **24%** of the useful context, and **0%** of the quantities that are arithmetic over what is already there — share of total, concentration, share of change. None of it was lost in transit: the whole evidence array is on the wire beside the artifact specs. A bar’s *width* encoded a share that no number ever stated, `slice(0,8)` dropped the tail in silence, and `means` — the one sentence saying what the figure actually is — reached the browser on every step and was rendered nowhere.

Two rules in that file. Nothing is authored by the model: totals and shares are arithmetic, and every number a model writes is a number that can be wrong and then has to be grounded. And a field that cannot justify itself is *absent*, not null — a renderer must never test eight nullables per row, and must never print “— %” where a share could not be computed.

Progress is real. The panel used to show a canned three-line rotation on a timer with no relationship to the work, so a two-minute investigation looked like a hang. The loop already knows what it is measuring and what it has ruled out, so it says so: typed events (`phase / objective / step / confirmed / rejected`), never a tick the UI has to sniff out of a string.

3.17 Cost, latency and limits

Quantity	Measured	Note
Model calls per question	median 7, mean 11	3 in the common case; each query step adds one, plus one per repair
Tokens	~8k in, ~1k out per call	the static context dominates
Latency, a figure	3–6s	one plan, one step, one write-up
Latency, an investigation	22–32s typical	this is model latency, not loop overhead
Latency, worst observed	~114s	seven sequential model calls at ~13s each
Write-up	5s median, 10s worst	measured across every logged trace, on any job
Cold knowledge build	60–70s, ~60 queries	a warm rebuild is far cheaper
<code>MAX_ROUNDS</code>	12	emergency ceiling, not a target
<code>MAX_STEPS</code>	40	
<code>MAX_CALLS</code>	30	counts repairs, not just generations
<code>MAX_MS</code>	180,000	checked <i>before</i> dispatching a step
<code>ROUND_RESERVE</code>	15,000	1.5× the worst observed write-up

Hitting any ceiling is a *failure* mode the answer has to disclose, not a normal finish. The analytical stopping condition is whether the question has actually been established.

What the paid suites cost. About \$1.10 for adversarial, \$0.50 for regression, \$0.20 for calc — under \$2 for all three. Worth stating because the instinct is to treat the benchmark as expensive and put it off. An unverified build is far more expensive than two dollars.

3.18 The test estate

FREE — `npm run pulse:check`. No model calls, no browser, a few minutes.

`pulse-view` 115 — prompts the model is given, evidence→view derivation, renderer admissibility, bindings, grounding, convergence, SQL reading, row identity

`pulse-deterministic` 65 — scoped metrics and ratios, repeatability, compute and its basis gate, routing, contract checks, `verifySpec`

`pulse-judge` 12 — *the benchmark's own judgement*, both directions: it must flag the bad answer and stay silent on the good one

`pulse-frontend` 7 — all 29 adversarial questions resolved (terms, families, scopes, windows) without answering them

`pulse-steps` — the steps the model ALREADY chose, re-run against today's code. The number that must stay 0 is "worked then, FAIL now"

`pulse-sql` — every complete generated query in the traces, replayed through today's validator, spec check and database

PAID — and these are the ones that matter.

`pulse:chain` 12 — "name him" lands on the SAME row the first answer named · an unresolvable reference asks instead of guessing · the CT payback reaches a number AND never takes commission from the payout ledger

`pulse:artifacts` 30 — does a card earn its place (none / earned / asked), and does the one that ships carry context, means, a total, shares and a sized tail

`pulse:bench` 29 — the adversarial headline · `pulse:regress` · `pulse:calc`

Last complete measurement. chain 12/12 · adversarial 28/29 · pure layers 115/115 · deterministic core 65/65 · judge 12/12 · frontend 7/7

Why the paid suites are the ones that matter

A prompt can say the right thing while the pipeline does the wrong one. `identityOf` was correct for weeks while "name him" answered about a different patient; every CT operand was measurable while the answer said it could not be established. **You can test a lot of individual gears and still not know whether they stay connected.**

One operational rule, learned expensively. Run one suite at a time. Every suite builds the knowledge index on startup, probing each registry metric against the database, and each run holds its own Prisma pool. Three at once exhausted the pool on the shared Neon instance: the knowledge build went from 80 seconds to **47 minutes**, eleven metrics reported **Timed out fetching a new connection**, and the analytics database stopped answering until the runs were killed. That is the production database.

Chapter 4

Limitations, and where we need to improve

This chapter is written against the system as it stands, not against a plan. Each limitation is stated as what actually happens, why it happens, what it costs, and what would fix it. Severity is judged on one axis only: *can this produce a wrong number the owner believes?*

	rare	occasional	routine
a silent wrong number	L7 materiality	L2 SQL not repeatable L6 regexes	L3 scope binding L5 grain not carried
a visible failure	L10 cold build	L4 eight dimensions L12 latency ceiling	L1 no degraded mode
friction	L11 multi-tenant	L8 usefulness L9 variance	L13 frontend L14 no action layer

4.1 L1 — there is no degraded mode

What happens. Every turn needs at least three model calls. When the provider returns 402, 401 or 429, Pulse produces nothing at all — not a worse answer, *no* answer. This has already happened twice mid-benchmark and once in a working session.

Why. The architecture is correct about the model’s role — it plans, interprets and writes, and none of those have a deterministic substitute. But seventeen of the eighteen tools are deterministic, and several of the most common questions (“how much did we collect today”, “due branch wise”, “top doctors last month”) map onto a single registry call with arguments the semantic index could resolve without any model at all.

What it costs. The refusal is now honest — it says the problem is the service and not the question — but honesty is not availability. An owner who opens Pulse to check today’s collection and is told the model is unreachable has been failed by a system that could have answered from a lookup.

Improvement.

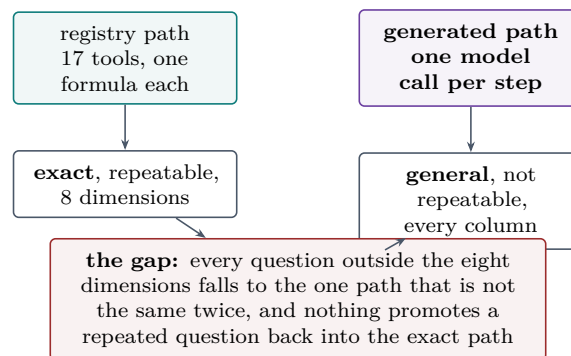
1. A **zero-call fast path**: when `scopeTermsIn(q)` resolves every scope word, a metric is unambiguous from the glossary, and a period parses, run the registry tool directly and write the sentence from a template. Measure what fraction of real logged questions this covers before building it — the answer is probably 25–40%, and if it is under 15% the path is not worth the surface.

2. A **second provider** behind 11mJson, chosen on failure rather than on cost.
3. **Cache the last answer per (question, day)**, so a repeat of today's question during an outage returns this morning's figure, clearly stamped with when it was measured.

4.2 L2 — generated SQL is not repeatable

What happens. The registry is exact and gives the same number every time. *query* does not. The same question has produced Rs. 83,700 and Rs. 1,03,400 on consecutive runs; a commission share has produced 21.4% and 18.4%.

Why. Each named cause has been fixed — rate versus total, test grain, ledger commission, denominator basis, the unreferenced CTE, the alias mismatch. Every one of those is a *named class*. The mechanism that produces them — a model choosing between two defensible readings of a grain — is not fixed, and the next unnamed class will arrive the same way.



Improvement.

1. **Mine the trace for recurring shapes.** The trace already records (question, spec, SQL, summary) for every turn. A weekly job that clusters questions by resolved spec and finds the ones asked more than n times, with stable SQL, is a list of candidate registry metrics — written once, by hand, and exact forever after.
2. **A repeatability gate on money.** Run a money question's generated SQL twice within the turn is useless (same SQL, same answer). Running *generation* twice and comparing the *figures* is the 2-sample agreement detector from Era 0, which catches ~36% at 5–10% false alarms. It was rejected as a *repair* signal — correctly — but it was never evaluated as a *disclosure* signal on money questions only, where the cost of a silent 12% error is highest and the extra call is affordable.
3. **Make the grain explicit in the spec** (see L5) so that a grain disagreement is caught structurally instead of by a named repair.

4.3 L3 — binding is Phase 1 only, and Phase 2 is the whole point

What happens. Time is bound forward: the spec resolves “last month” to two IST dates and hands them to the generator as an authoritative clause. **Scope is not.** A scope constraint is resolved upstream, discarded, re-derived from English by the generator, and then *recognised* backwards by a verifier.

Why it matters more than it looks. v2/spec.ts contains this:

```
const COLUMNS: Record<string, RegExp> = {
  domain: /\b(v|visit)\."?domain"?|"?domain"?s*(=|in)/i,
  branch: /\bbr\."?(code|id)"?"?Branch"?|b|"?branchId"?/i,
  payment_type: /"?paymentType"?/i,
  referring_doctor: /"?ReferralDoctor"?|"?referralDoctorId"?/i,
  test: /"?testCodeSnapshot"?|"?testNameSnapshot"?|"?testDefinitionId"?/i,
  ...
};
```

That is a regex table mapping a semantic dimension to the SQL shapes it might appear as. It is the exact structure this architecture spent two days deleting from renderer selection and prose contracts, surviving in the one file whose job is to make scope reliable. It has already been patched four times — aliases, LIKE breadth, derived-dimension sets, outer joins — and each patch teaches the verifier one more *dialect* of a question that was already answered upstream.

What it costs. Every scope failure in Chapter 2 — the Chintal total, the imaging commission, the CT non-determinism, the “external reports” refusal — lives downstream of this one decision.

Improvement. This is the highest-value structural work available.

1. Extend QueryBinding with kind: `'scope'`, carrying the *predicate itself*: the DIMS expression and the literal(s), already joined-and-cased, e.g. `br.code IN ('CNT')` or the twelve-line `modality CASE` with its value.
2. Emit it into the generator’s context as a clause it must include verbatim, in last position next to the question, the same place the time binding already works.
3. `verifySpec` then checks for the *binding it emitted*, not for a dialect it has to recognise — which collapses `COLUMNS`, `DERIVED`, `DERIVED_SETS`, `aliasesFor` and most of `verifyConstraint` into one string comparison.
4. Keep the parse-and-walk check (§2.9). It answers a different question — *does the predicate restrict the result* — and the unreferenced-CTE failure is not addressed by binding.

Why it was deferred, and why that was right at the time

Phase 1 was deliberately time-only because time has no joins, no aliases and no derived expressions: the cheapest possible test of whether a typed binding survives the generator at all. It does. The mechanism is proven and the reason for the deferral has expired.

4.4 L4 — the registry filters on eight columns; the schema has hundreds

What happens. DIMS has eight entries. Any question narrowed by anything else — `workflowMode`, `paymentStatus`, a department, a report status, an external lab, a coupon — cannot use a registry tool and falls to generated SQL.

Why it is not merely a gap. The system spent a whole fix (d99bc35, “the layer denied the existence of one of its own dimensions”) on the consequence: “how much am I making from external reports” resolved *correctly* to `EXTERNAL_UPLOAD`, a `workflowMode` on `TestOrder` with 1,400 orders behind it, and then died in `buildFilter` because `workflowMode` is not one of the eight. The analyst read the refusal as *absence* and told the owner the figure could not be established.

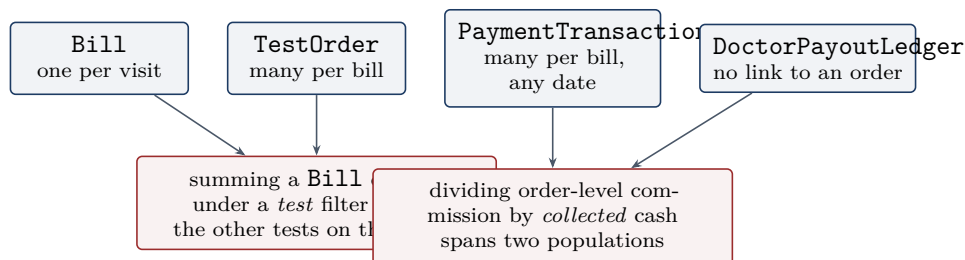
The error message now says in as many words that this “says nothing about whether the data exists”, and `registryFilterable: false` carries the constraint forward. Both are mitigations for a structural shortfall.

Improvement. Derive DIMS rather than authoring it. Every column that is an enum, a low-cardinality text column, or a foreign key to a small dimension table is a candidate dimension; the build already reads column types, row counts and enum labels. The hand-written entries would shrink to the three *rolled-up* ones (*modality*, *service_kind*, and the doctor join), which are genuine business judgements and belong in code.

4.5 L5 — grain is enforced by name, not carried as data

What happens. `comparable()` compares period, dimension, scope, metric and unit. Two figures with matching identity strings can still be about different populations: bill-level versus order-level versus payment-level versus ledger-row, all within “August, CT”.

Why this is the deepest remaining correctness hole. It is the class that produced the CT ROI failure, the imaging commission failure, the denominator-basis failure and the rate-versus-total failure — four separate incidents, each currently prevented by a *specifically named* repair inside `t_query` or a *specifically named* guard inside `runStep`. There is no general rule. A fifth population mismatch will not be caught.



Improvement. Add grain to `Evidence` and to `Metric` — an enumerated value (`bill`, `order`, `payment`, `ledger`, `visit`, `report`, `result`) derived from the `FROM` clause of the SQL that produced it, which `sqlscope.ts` already parses. Then:

- `comparable()` refuses across grains by default, so `compute` cannot form the ratio at all.
- A scope constraint on `test`, `payout_category`, `modality` or `service_kind` requires grain `order`, structurally — which is the general form of `TEST_GRAIN` and `COMMISSION_BASIS` together.
- The named repairs stay as *hints* for the repair prompt, but stop being the only thing standing between the owner and a plausible wrong ratio.

4.6 L6 — roughly a dozen regexes remain in load-bearing positions

What happens. The architecture’s central rule is that nothing may match words against the question to decide how to answer. Two ladders were deleted for violating it. These remain:

Pattern	Decides	Has already been wrong
PRONOUN_ONLY	whether to ask instead of answer	fired only after the fragment rewrite, so “name him” named the wrong person
FRAGMENT / DIM_PHRASE	whether a short line inherits the last question	“doctor wise” after “branch wise” produced a cross-tab
SHOW / REFUSED	whether an artifact ships	“no table” shipped a table
QUALIFIED	whether a registry step is rewritten to query	
rateQ	whether a total is really a rate	“give me cost” is variance-prone today
testScoped / workScoped	which money basis is legal	workScoped on the <i>question</i> is far too broad for refusing a metric; narrowed to the spec in c780871
DISCLOSED	whether incompleteness was admitted	
PCT + “discount”	the impossible-share check	“1,250%” escaped by matching only “250” after the comma
month/period matcher	the period-grounding check	“may” fired on “concepts that may appear”
ordinary() rule	date grounding false positives	a date read as an invented figure

Why they are still there. Each is genuinely small, and each sits at a *boundary* where there is no structured input yet — the raw sentence at ingress, or the finished prose at the exit. That is a real difference from the deleted ladders, which sat in the middle with structured data on both sides. But the difference is one of degree.

Improvement. For each, move it to one of two places:

1. **An explicit field the analyst must fill.** **present** already did this for renderer preference and it worked — the model resolves the sentence, the code never sees it. **askedToSee** / **refusedToSee** and **rateQ** are the obvious next three.
2. **A structural check on resolved values.** The impossible-share check should not look for the word “discount” near a percentage; it should compare the two grounded facts the percentage was *derived* from and refuse when a part exceeds its whole. Grounding already knows which facts produced the figure.

4.7 L7 — materiality is the model’s own label on its own hypothesis

What happens. **openMaterial()** filters hypotheses by **h.material**, a boolean the model set when it wrote the hypothesis. The loop continues while something material is open and stops when nothing is. A model that under-labels materiality stops early, and the answer then looks complete because nothing flagged it as otherwise.

Why it survived. It is a large improvement on the boolean it replaced — **enforce()** still requires the named analysis to have *run*, so a claim cannot be settled on assertion alone. The gap is not “is this claim proven” but “should we have cared”.

Improvement. **isMaterial(value, total, largest, question)** already exists in **contract.ts** and is applied only to prose. Apply it to hypotheses: once a hypothesis has any evidence attached, its materiality becomes a *measured* question — is the quantity it is about large enough, against

the headline, to change what the owner does. A claim about 0.4% of the total is not material regardless of what the model called it, and a claim about 30% is material regardless.

4.8 L8 — nothing measures whether the answer was useful

What happens. The contract check, the grounding check, the scope check and the period check all ask *is this defensible*. Nothing asks *is this what the owner needed*. The artifact-fit suite (30 cases) is the only approximation, and it is hand-written — it encodes the author’s judgement of what a good card is, not the owner’s.

The signal that exists and is unused. Every question is audited, and the audit log contains the *sequence*. A follow-up that narrows, re-asks or corrects is direct evidence that the first answer missed. “branch wise” immediately after a total means the total was not what was wanted. “no, in rupees” means a unit was wrong. “name him” means a card should have carried an identity.

Improvement. A **follow-up taxonomy** mined from the audit log: classify each consecutive pair as *drill*, *correct*, *repeat*, *new*. *Correct* and *repeat* are defects. This is a weekly report, not a test, and it would be the first measurement of usefulness the project has ever had.

4.9 L9 — run-to-run variance is no longer quantified

What happens. The adversarial suite reports a single number — 28/29. Era 0 measured run-to-run noise at ± 2 questions on a 32-question set, or roughly $\pm 6\%$. We have not re-measured it on the current suite. A movement from 28 to 26 could be a regression or could be noise, and there is no basis for telling them apart.

Why it matters now more than then. The suite is much harder than it was and the failures are concentrated in the questions with the most machinery behind them. Exactly one failure is currently outstanding (“give me cost”, `WRONG_GRAIN`) and it was verified as *variance* by reproducing it in isolation, where it answers correctly. That single observation is the whole evidence base for the claim.

Improvement. Run the adversarial suite **three times** at the next funded opportunity and report the mean, the range, and the per-question stability. \$3.30 buys the noise floor, without which no future score means anything.

4.10 L10 — the knowledge build runs against production

What happens. A cold build issues roughly 60 queries against the live clinical database — enum labels, every column in the schema, row counts per table, fill rates on a watch list, the value index across five tables, and one probe per registry metric. It takes 60–70 seconds and happens on every process start.

What it has already cost. Three suites launched in parallel exhausted the Prisma pool on the shared Neon instance. The knowledge build went from 80 seconds to **47 minutes**, eleven metrics reported `Timed out fetching a new connection`, and the analytics database stopped answering until the runs were killed. Neon branches are data-isolated but *not* quota-isolated, so a test branch does not help.

Improvement.

1. **Persist the build**, keyed by the fingerprint that already exists. A cold boot reads it; only a fingerprint change rebuilds. This turns 60 queries per process start into zero for the common case and makes the harnesses free of the database entirely.
2. **Sample `checkRegistry`** rather than running all fifteen formulas on every build; a rotating subset, with the full sweep on `/health`.
3. **Cap the pool per process** so a harness cannot starve the portal.

4.11 L11 — everything in the knowledge layer is Sobhana’s

What happens. The glossary names this centre’s words. The ontology encodes this centre’s relationships. `TEST_BRANCHES` names two specific branch codes and one specific string, “(Kidcare)”. `DUE()` encodes this centre’s four discount and reversal columns. The fifteen metric formulas are hand-written against this schema.

Why it matters. The stated direction is Axora / HealthFlow — the same portal serving many diagnostic centres. A new tenant today would need a developer to write a glossary, and would inherit another clinic’s test branches.

Improvement. Split `catalog.ts` and `knowledge.ts` along the line that already exists in the code but is not enforced:

- **Schema-derived** (graph schema, value index, coverage, grants, enums, fingerprint) — already tenant-neutral, works on any HealthFlow database unchanged.
- **Tenant-declared** (glossary synonyms, test branches, metric definitions that encode a house rule such as “revenue means cash collected”) — becomes a stored, editable object with a UI, versioned per tenant.
- **Product-invariant** (join and grain conventions, the paise rule, the IST rule, the cancelled-row rule) — stays in code, because it is a property of the schema the product ships.

4.12 L12 — the latency ceiling is honest but it is still two minutes

What happens. A question like “why is the business down” can take up to 180 seconds and then disclose that it stopped early. That disclosure is the right behaviour and it does not make the wait shorter.

Why. The cost is seven or more *sequential* model calls at ~13s each. A per-step timeout was tried and removed — 114s with it, 114s without — because the sequence, not any one step, is the cost. Parallelism within a round already exists (pool of 3) and does not help, because the rounds themselves are sequential by construction: each depends on what the last one learned.

Improvement. Two directions, neither cheap:

1. **Speculative round one.** For explanation-shaped questions, dispatch the standard decomposition (by branch, by domain, volume versus realisation, by referrer, by service mix) *before* the investigate call returns. Those are registry tools, so they are free; the investigate call then reads five results instead of one and often needs no second round.
2. **Stream the answer.** Progress lines already stream; the answer arrives whole. Streaming the verdict sentence first, then the points, would make a 40-second investigation *feel* like a 10-second one without changing a single number.

4.13 L13 — the frontend is the least-tested surface

- `PulseArtifacts.tsx` is 402 lines and largely one switch. Adding a renderer is a capability entry plus a case, which is the intended design, but the file is now at the size where the cases start sharing helpers by copy.
- A `worklist` can return 200 rows and the table is not virtualised.
- `tsc` and `vite` build both pass on conditional hooks and on a double paise conversion. A waterfall showed Rs. 2,750 as +**Rs. 28** while every check was green. The lesson is recorded and the only defence is looking at the rendered card.
- Verification is by a DEV-only gallery route rendering real renderers against fixture evidence. That is good, and it is manual.

Improvement. Snapshot the gallery. The fixtures exist, the route exists; rendering each artifact type at three widths and diffing the DOM would convert a manual look into a free check.

4.14 L14 — Pulse answers, it cannot act

The `opportunity` job ranks levers by honest impact and says what each is worth. The owner then does the work by hand, in another part of the portal. There is no path from “these 27 patients owe Rs. 7,002 and 11 of them have a phone number” to a WhatsApp campaign, even though the portal has a campaigns module. This is a product boundary rather than a defect, but it is the largest gap between what Pulse knows and what it changes.

4.15 L15 — known open issues in the data itself

These are facts about the business, not bugs in Pulse, and they limit what any answer can mean.

payoutCategorySnapshot is NULL before 1 Aug 2026 The payout ledger and a recomputation disagree	24.5% of July. Any scan or modality count reaching into July is understated and must say so. Currently it does not. August referral commission has three defensible answers: recomputed from orders Rs. 4,97,868, naive Rs. 5,29,871, stored ledger Rs. 5,34,722 — up to 7.4% apart, 12% on one doctor. Some doctors match to the rupee, so it is a population or period mismatch, not a rule difference. Unreconciled.
Discount reasons fragment by case NOREFARAL is offered as a person	SELF / self / Self are three rows in every breakdown. A sentinel value in the referring-doctor column reaches rankings as a name.
Duplicate doctor records	“Dr.K.RAMASWAMY MD” and “K.RAMASWAMY” are two rows, so one doctor’s volume splits across two names.
The database starts 1 July 2026	There is no pre-software baseline . Any year-on-year or “since we started” question is unanswerable and should be refused rather than approximated.

4.16 What to do next, in order

Ranked by expected reduction in silent wrong answers per unit of work.

#	Work	Size	Why first
1	Carry grain on evidence and metrics (L5); make <code>comparable()</code> refuse across grains	1–2 days	Generalises four separate named repairs into one structural rule. The deepest remaining correctness hole.
2	Phase 2 binding — scope (L3)	2–3 days	Deletes the last regex table from the correctness path and collapses half of <code>verifySpec</code> . The mechanism is already proven on time.
3	Three adversarial runs for a noise floor (L9)	\$3.30	Every future score is uninterpretable without it.
4	Persist the knowledge build (L10)	1 day	Removes 60 production queries per process start and makes every harness database-free on the build path.
5	Follow-up taxonomy from the audit log (L8)	1 day	The first measurement of usefulness this project has ever had, from data already stored.
6	Zero-call fast path, measured first (L1)	0.5 day to measure	Do not build it until the log says what fraction of real questions it covers.
7	Move <code>askedToSee</code> / <code>rateQ</code> onto the plan (L6)	0.5 day	Two of the highest-traffic regexes, both already wrong once.
8	Materiality from magnitude (L7)	0.5 day	<code>isMaterial</code> already exists; apply it to hypotheses.
9	Derive DIMS from the schema (L4)	2 days	Widens the exact path; only after grain is carried, or it widens the wrong path.
10	Snapshot the artifact gallery (L13)	1 day	Converts the one manual check into a free one.
11	Split tenant-declared from schema-derived knowledge (L11)	3–5 days	Blocks Axora, not Sobhana. Do it when a second tenant is real.

The one thing not on this list

No item above adds a mechanism that asks the model to check its own work. Sixteen days of measurement said that never moves anything, and every hour spent on it is an hour not spent on grounding. **If a future change is tempted back in that direction, the burden of proof is a measurement, not an argument.**

Appendix A

Reference tables

A.1 The metric registry

Fifteen formulas. Each is the *only* place that expression exists in the codebase, and each encodes a convention the schema cannot imply.

Metric	Unit	Splits by	The convention it carries
revenue	paise	branch, payment_type, referring_doctor, domain	Net cash <i>received</i> in the period, refunds netted out. The house definition of revenue — never billed amounts
net_billed	paise	branch, domain, referring_doctor	Billed after discount, coupon and voided charges. <i>Not</i> revenue
outstanding	paise	branch	Net billed minus paid, on open bills — the computed balance, never the paymentStatus flag
visits	count	branch, domain, referring_doctor	Registrations. One per visit, not per person
unique_patients	count	branch, domain	Distinct people, deduped across repeat visits
test_orders	count	branch, test, category, modality, service_kind, doctor	Billed test lines, <i>excluding cancelled</i>
reports_finalized	count	branch	Finalized report versions, latest per report only
tat_p50	minutes	branch	Median minutes registration to finalize. SLA 1440
abnormal_rate	ratio	—	Denominator excludes flag IS NULL — 38% of rows
cancellation_rate	ratio	—	Share of test orders cancelled
delivery_rate	ratio	—	Delivered or read, channel = WHATSAPP only
discount_total	paise	branch	Manual discount plus coupon
refund_total	paise	branch	Money returned to patients
billed_on_orders	paise	branch, test, category, modality, service_kind, doctor	Order-level billing. Exists so a <i>scoped</i> ratio has a denominator at the same grain
commission_on_orders	paise	branch, test, category, modality, service_kind, doctor	The commission <i>frozen on each order</i> — percentage of price, or a flat amount
commission	paise	referring_doctor, branch	What a doctor was <i>paid</i> , from DoctorPayoutLedger . Carries no link to a test order and may never be scoped to work

A.2 The eight filterable dimensions

Dimension	Expression	Note
branch	br.code	JGG and IDPL scrubbed from splits
domain	v.domain::text	DIAGNOSTICS is the lab side; CLINIC is OP/IP
payment_type	pt."paymentType"::text	only reachable from PaymentTransaction
referring_doctor	COALESCE(rd.name, '(none)')	the doctor who <i>sends</i> the work, never the one who sees the patient
test	o."testCodeSnapshot"	a test is both a code and a name; both are aliases of one identity
payout_category	o."payoutCategorySnapshot"	raw, and NULL before 1 Aug 2026
modality	rolled-up CASE	folds Ultrasound / Ultrasound Tiffa / 2D Echo into one; X-Ray / Dental X-Ray into one
service_kind	rolled-up CASE	IMAGING versus LABORATORY

Appendix B

Catalogue of named failures

Every entry below actually reached an owner or a benchmark, and every one is now closed by code. The column that matters is the last: what class of thing went wrong.

What the owner saw	Cause	Class
Chintal billed 14,111 tests	a registry tool silently dropped a filter it could not express	scope dropped
“Lab collection” equal to the centre total Rs. 18,61,499 of CT billing in August	the domain qualifier never reached the query trend accepted the filter and built its own WHERE without it	scope dropped scope <i>accepted</i> , <i>not applied</i>
“21.4% over the last 90 days” from a 30-day step	the period came from the plan, not the query	period unverified
“At the current CT run-rate of Rs. 18,93,725” Rs. 8,69,235 for one test	a constraint too weak to filter on was used to label the spec guarded SQL and nothing guarded registry steps	scope claimed, not applied guard asymmetry
A confident 0 for ultrasound	the validator <i>forced</i> a filter matching nothing	unresolved value trusted
CT commission 4.5× CT revenue	DoctorPayoutLedger summed under a test scope	wrong population
Commission share 18.4% against a true 21.4%	order-level commission over collected cash	two populations
Rs. 83,700 and Rs. 1,03,400 on consecutive runs	bill grain versus order grain	grain non-determinism
“A CT-BRAIN PLAIN is sold at Rs. 1,03,400” 435,854,453	a sum across 47 orders labelled as a rate	rate versus total
A payback of 0.5 months	raw paise reached the writer	unit lost in transit
“I could not establish it” with every operand present	a generated query never declared its unit no place for arithmetic to happen	unit undeclared missing capability
Nulls on every step after “calc”	“calc” bound to CAL , a lab test code	resolution without context
A CT question answered about Clotting Time	three-letter substring match outranked the category	resolution without family
“The centre does not record patient names”	a hardcoded grant list had drifted	declared, not derived
“There is no field that records mistakes”	a lexical miss read as a schema fact	absence versus ignorance
“CT-scoped revenue has never been measured”	a refusal that named no alternative	dead-end error
“No patients currently have outstanding dues”	a guard rejected every operational tool	guard too broad
10 debtors owing Rs. 5,002 against a true 11 / Rs. 5,802	the paymentStatus flag instead of the balance	wrong column
5,104 “scans” against a true 528	“scan” undefined, collapsed to every test order	undefined term
“1–11 September” missing that day’s Rs. 15,650	an exclusive end date, unmarked	boundary
A waterfall showing Rs. 2,750 as +Rs. 28	double paise conversion in the renderer	unit, again
“Discounted at 1,250% of what it bills”	bill-level discount over one test’s price	impossible share
A split question drawn as a time series	a regex ladder matched “last 30 days” first	words deciding the answer
“Which mbbs?”, twice	entity resolution ran before conversation reference	precedence
“Name him” naming the wrong patient, then a doctor, then a staff member	the subject was not bound to the row	reference unbound
A KPI tile under “just tell me, no table”	the negation matched the request pattern	words, again
A titled empty card	structure satisfied, renderer input absent	admissibility by name
A 2-minute investigation that looked like a hang	canned progress on a timer	instrumentation
7 of 24 clean	seventeen billing failures scored as quality failures	instrument, not product

What the owner saw	Cause	Class
19 of 26 answers flagged as invented Nine rounds, thirty-three calls, a worse answer	the trace stored row counts, not values a contradiction the pipeline invented, re-measured	instrument no dedupe, no applied-scope check

Read down the last column and three classes account for almost all of it:

1. A scope, period, unit or population that was asked for and not applied — and a check that looked at the request rather than the result.

2. A limitation of Pulse reported as a fact about the business — an index miss, a missing grant, a registry gap, a policy rule.

3. Words deciding something that should have been decided from structure.

Every architectural decision in Chapter 3 is aimed at one of those three.

Appendix C

File map

File	LOC	Responsibility
Backend — src/services/pulse/		
index.ts	132	Ingress, precedence gates, small talk, follow-up rewriting, refusal classification
knowledge.ts	928	Every static block the model is given; the live build of graph schema, value index, semantic index, coverage, grants; term resolution and family disambiguation
catalog.ts	196	The metric registry, dimensions, FROM clauses, test branches, the dues expression
db.ts	72	The analytics_ro client, row normalisation, the fan-out pool
llm.ts	65	The single model entry point
validator.ts	96	Deterministic SQL safety and the row-level policy
sqlPath.ts	35	Context assembly and one generation call
diagnostic.ts	96	Periods, baselines, window labels
entity.ts	68	The zero-call entity card
shapes.ts	15	Metric guessing for state continuity
today.ts	38	The empty-state pack
Backend — src/services/pulse/v2/		
tools.ts	1045	The eighteen tools, the guard cascade, the typed repairs, the writer projection
run.ts	585	The orchestrator: rounds, budgets, dedupe, ordering, artifact judgement, the trace
analyst.ts	490	The three prompts
spec.ts	398	AnalysisSpec , completeSpec , verifySpec , lineage, scope recheck
sqlscope.ts	387	Parse and walk: effective predicates, exposed identifiers, group keys, period and filter reading
investigation.ts	366	Hypotheses, enforcement, progress measurement, stagnation, comparability, contradiction screening
contract.ts	344	The response contract per job, checkAnswer , simplify
capability.ts	292	describeEvidence , the ten renderer capabilities, the matcher
calc.ts	217	The restricted evaluator and unit normalisation
artifacts.ts	188	Turn artifacts, reference resolution, row identity
grounding.ts	181	Facts, derivation, ordinariness, the ungrounded list
opportunity.ts	111	Ranking levers by honest impact
binding.ts	97	QueryBinding — time, Phase 1
Frontend — health-hub/src/components/pulse/		
PulseArtifacts.tsx	402	The ten renderers
PulsePanel.tsx	292	The feed, the progress stream, the evidence drawer
deriveView.ts	145	Evidence to view: one derivation, shared
usePulse.ts	114	State, prefetch, the today cache
pulse.css	103	
PulseOrb.tsx	31	
format.ts	22	
Pulse.tsx	18	The owner gate

The two sentences this document exists to preserve

Grounding is the only lever that has ever moved this architecture; post-hoc verification has never moved it at all.

An invariant described in a prompt is not an invariant.